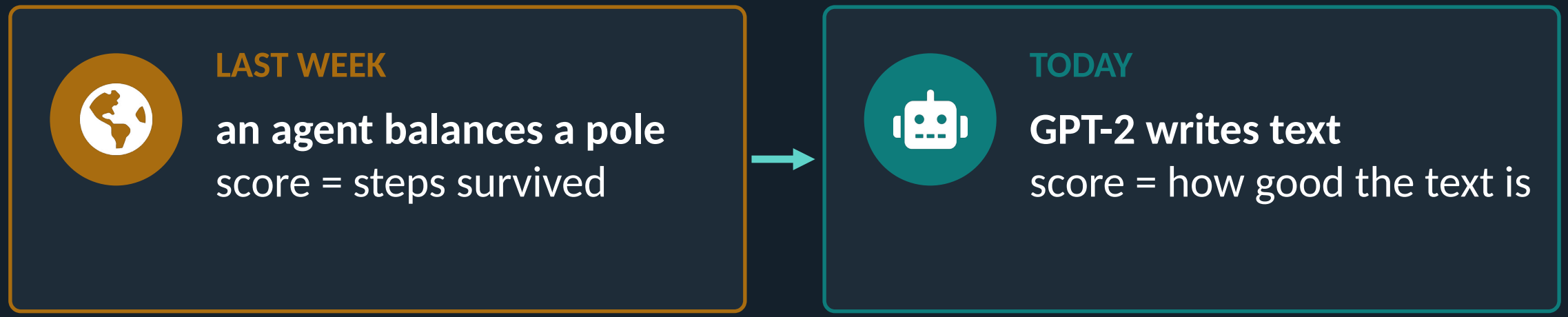
TARA · RL WEEK 2 · ARENA 2.4

RLHF: teaching GPT-2 with a score

The same RL loop as last week. The agent is now a language model.



TODAY · ~82 MIN · A PAUSE AFTER EVERY PART

OPENING

Seven parts, each built on the one before

1 · Revision

VPG → PPO's three fixes
11 min

2 · Why RL

pretraining → SFT → judging
8 min

3 · Reward

human feedback → a number
14 min

4 · Agent

GPT-2 as the agent
9 min

5 · Credit

which tokens earned it?
13 min

6 · KL

don't drift from GPT-2
25 min

7 · Train & LoRA

trainer, PEFT/LoRA, GRPO
25 min

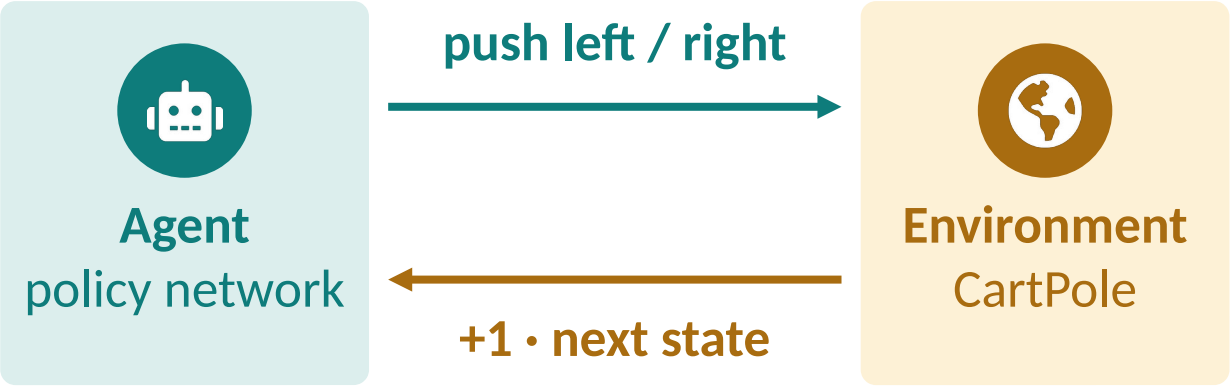
Slido: code in the Zoom chat · one question per part · Q&A in every pause

1 · REVISION · ~11 MIN

Revision: VPG, and PPO's three fixes

Only what this notebook uses.

The loop: the agent acts, the environment answers



The goal
find a policy whose games have a high **average return**: J(θ)

ingredient	CartPole
Agent	policy network
State	4 numbers (cart, pole)
Action	push left or right
Reward	+1 per step
Transition	physics moves the cart
Return	sum of rewards

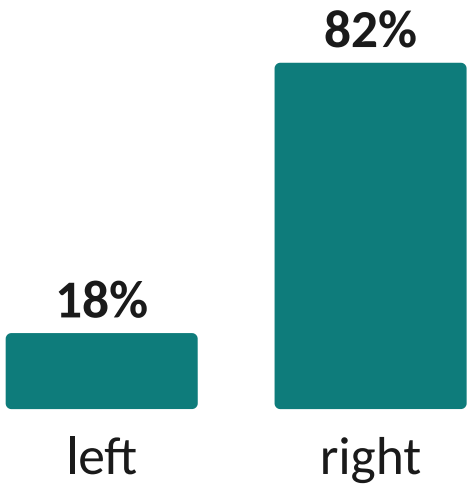
1 · REVISION

SKIP IF SHORT

The policy is a classifier whose classes are actions

state s
0.10, +0.50,
+3°, +0.20

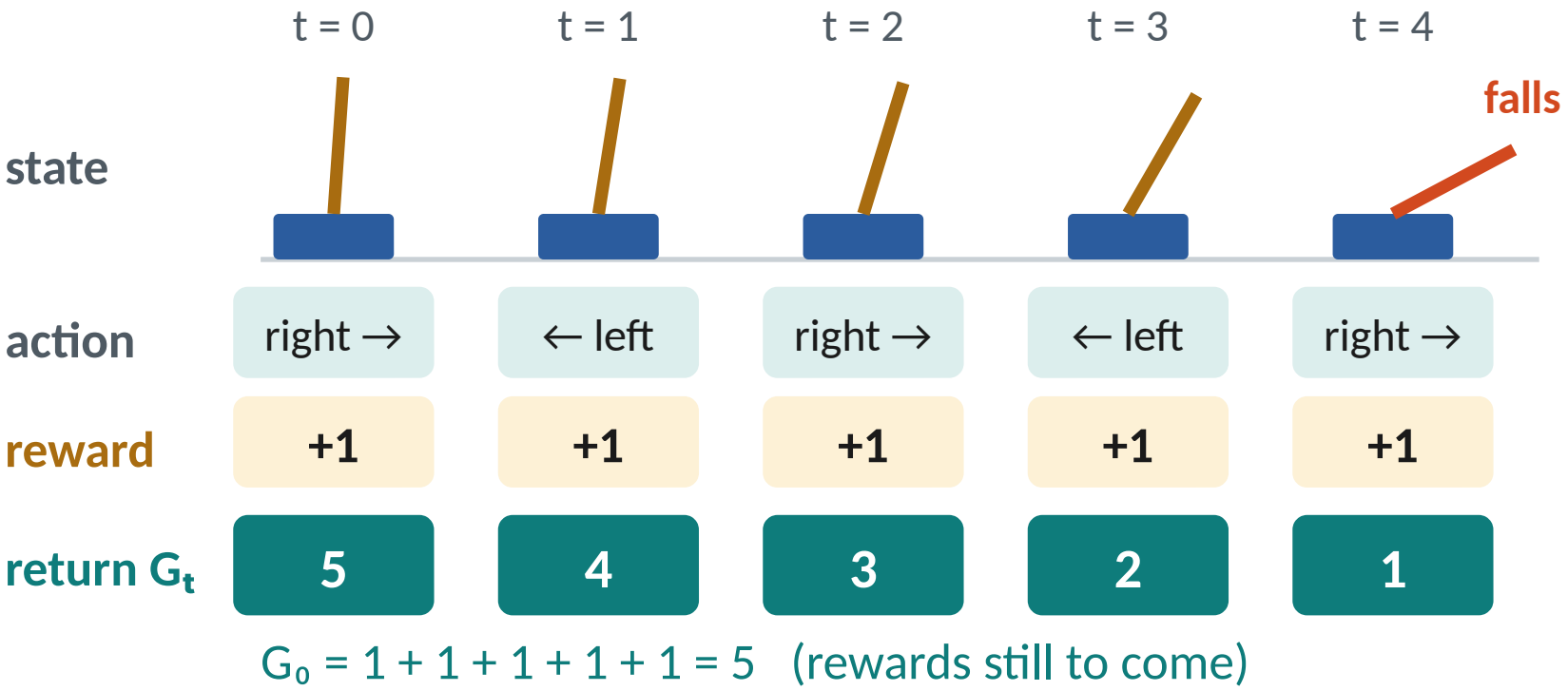




→ roll the dice: **right**

Learning = changing θ so that good actions get more probability.
Supervised learning is told the right class. RL only gets a **score** afterwards.

One game of CartPole: rewards, the return G, and the "usual"



$G_0 = 1 + 1 + 1 + 1 + 1 = 5$ (rewards still to come)

The "usual" from t = 0
other envs, same step:

G = 5	G = 40
G = 120	G = 15

average = **45**

(last week: 32 envs)

$score_0 = G_0 - usual = 5 - 45 = \mathbf{-40}$ → the push at t = 0 was worse than usual → make it **less likely**

VPG: climb the average return by gradient ascent

Goal: make J(θ), the average return of the games π_θ plays, as **big** as possible.

$$J(\theta) \approx \frac{1}{N}\sum_{i=1}^{N} G^{(i)}$$

1 · Play

32 games with the current π_θ
→ states, actions, rewards, G_t

2 · Score

$$b_t = \frac{1}{32}\sum_{i} G_t^{(i)}$$

$score_t = G_t - b_t$

3 · Step uphill

$$\nabla J \approx \text{mean}[\nabla \log \pi_\theta(a_t|s_t) \cdot \text{score}_t]$$

$\theta \leftarrow \theta + \alpha \nabla J$

4 · new θ → play brand-new games → repeat (40 rounds last week)

round 1: J ≈ **22**	round 10: J ≈ **61**	round 20: J ≈ **140**	round 40: J ≈ **500**

VPG has three problems; PPO fixes each one

PROBLEM IN VPG	PPO'S FIX
① **The "usual" ignores the state** upright and falling poles at step 100 get the same baseline	**Critic V(s)** learn the usual return for each state
② **Wasteful** 16,000 steps of play → one gradient step → thrown away	**Importance weights** reuse each batch for several steps
③ **Reuse can go too far** many steps on one batch move the policy far from the one that played	**Clip** stop pushing once a probability has changed enough

Fix 1: a critic network learns what is usual for each state

state s
4 numbers

→

critic
network

→

V(s)
one number:
usual return
from here



V ≈ 400



V ≈ 5

How it learns

after each batch we know the real returns G:
value loss = $(V(s) - G)^2$

How it is used

score A = G − V(s)
falling pole saved, G = 150 → A = **+145**

VPG's average (≈ 200 at that step) would have scored it **−50**. The critic knows the state.

Fix 2: reuse each batch, corrected by importance weights

Play one batch
with π_old: 32 envs × 500 steps

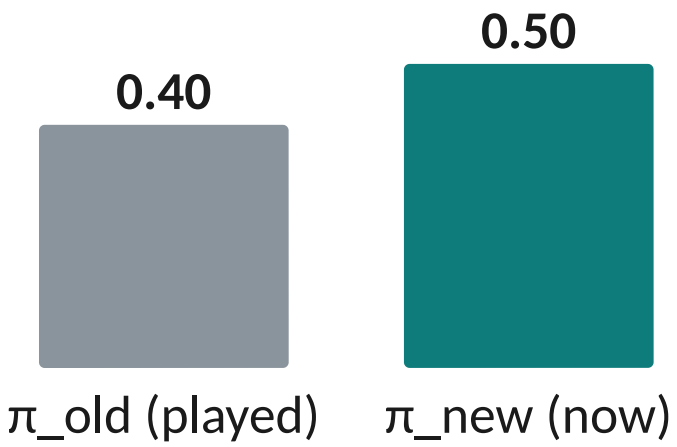
→

Update 1
π changes a little

→

Update 2, 3, …
same batch, but π is now different

One action from the batch: **push right** in state s



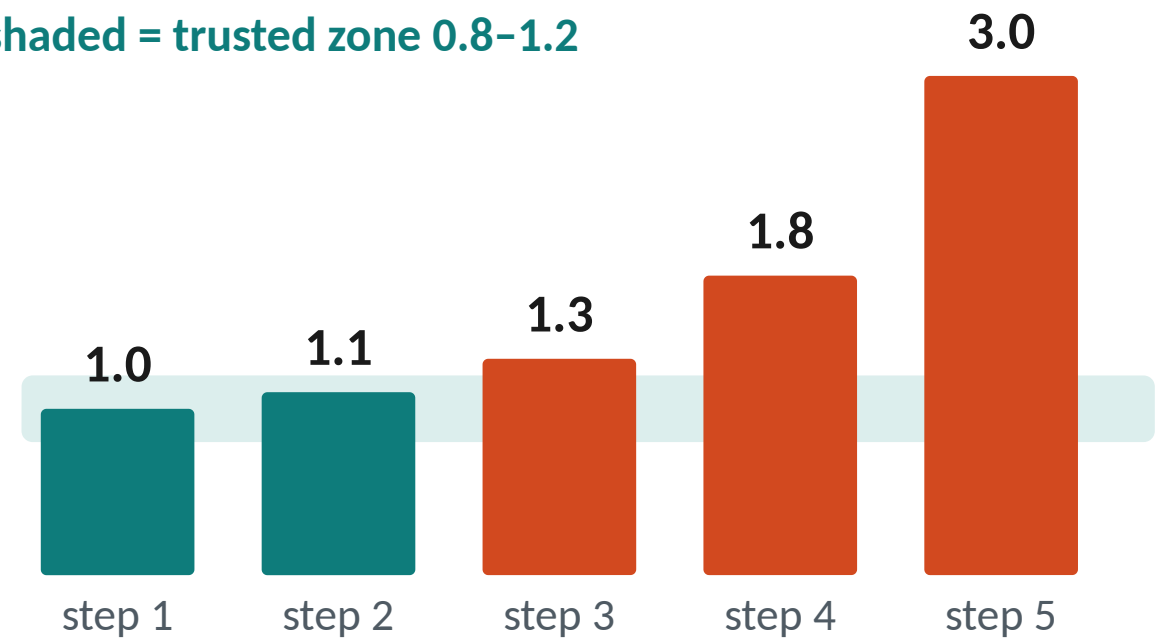
$$r = \frac{\pi_{\text{new}}(a \mid s)}{\pi_{\text{old}}(a \mid s)} = \frac{0.50}{0.40} = 1.25$$

Counts **1.25×**. Re-weighting the old games by r tells us how the **new** policy would score, without replaying.

Fix 3: clip, so one batch can't move the policy too far

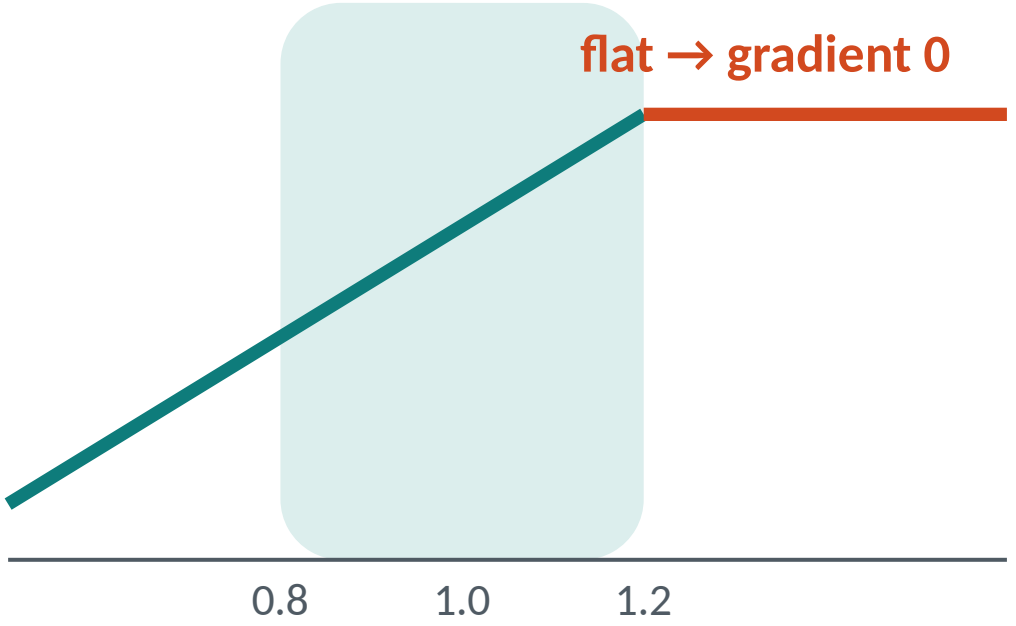
Same batch, more updates: r keeps growing

shaded = trusted zone 0.8–1.2



r × A keeps paying off, but the data never saw a policy that far → **overshoot**

gain for a good action (A > 0) vs r



Clip: new prob ÷ old prob may reach **1.2** (or **0.8** for a bad action). Past that: no more push.

PPO: collect a batch, then maximise three terms

Rollout play · store states, actions, log-probs, advantages → **Learning** several minibatch steps on the objective

+ clipped term

better-than-usual actions more likely

− value loss

the critic's forecasts improve

+ entropy

stay a little random

+ ?

one more term today

→ Part 6

Maximised by gradient ascent. **This notebook uses these same three** (clipped term and value loss are given).

LIVE DEMO · 07_VPG_PPO_RECAP.HTML

VPG and PPO, hands on

DEMO

SKIP IF SHORT

OPEN THE PAGE

▶️ Open demo 07

`/Users/jank/Downloads/tarapresenweek6/07_vpg_ppo_recap.html`

Tab 1: four games, the "usual", the scores, then a gradient step and new games. Tab 2: one action's importance weight and clip.

WHAT WE DO ON IT

1. Tab 1: move t; watch the usual and the scores.
2. Press "Gradient step" a few times: J climbs.
3. Tab 2: drag π_{new} past $1.2 \times \pi_{old}$: gradient 0.

SLIDO · QUESTION 1

SLIDO

An action has A = G − V(s) > 0. What does that mean?

A. The reward for that step was positive

B. The critic made a mistake

C. The action did better than what usually happens from this state

D. Its importance weight is above 1.2

Q&A

Pause: questions before we move on

WHAT WE JUST BUILT

1. VPG: play, score = G − usual, step uphill on J, repeat.
2. PPO: critic V(s); reuse a batch with importance weights.
3. Clip: at most 1.2× or 0.8× per batch.

DISCUSS

CartPole: state = 4 numbers, action = a push. For a **chatbot**, what would the state and the action be?

Slido Q&A is open
ask or upvote · ~2 min

2 · WHY RL · ~8 MIN

Why RL on top of a language model?

Pretraining gives a model that can write. Fine-tuning makes it answer. Judging makes it better.

Same recipe, new agent: from a pole-balancer to a chatbot

Last week · CartPole

agent	small policy network
action	push left / right
reward	+1 per step upright
learning	PPO

Today · GPT-2

agent	GPT-2 (124M weights)
action	the next token
reward	a score for the text
learning	PPO + one new term

TODAY'S QUESTIONS

1 · Why does a language model need RL?

2 · How does human feedback become a reward?

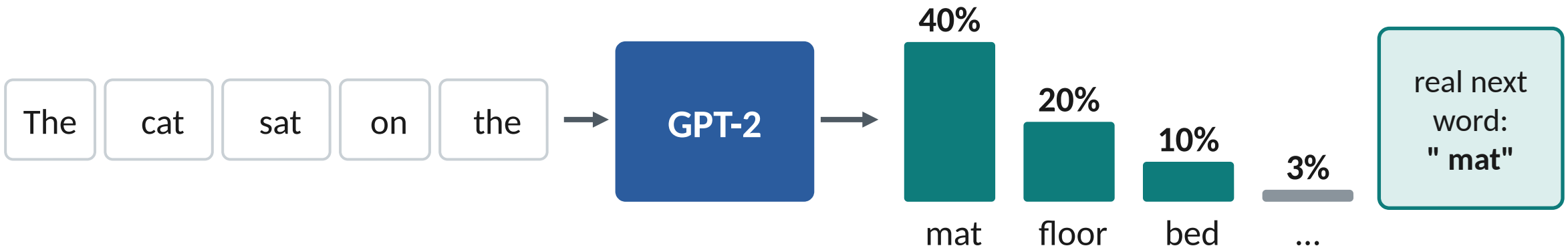
3 · How is GPT-2 an RL agent?

4 · Which tokens earned the reward?

5 · How do we stop it gaming the reward?

6 · How do we train it cheaply?

Pretraining: predict the next word, with labels for free



EVERY POSITION IS A TRAINING EXAMPLE

The → **cat**

The cat → **sat**

The cat sat → **on**

The cat sat on → **the**

THE LOSS

$$\mathcal{L} = -\log p_\theta(\text{ real next word })$$

= cross-entropy, as for any classifier.
" mat" got 40% → loss **0.92**
" mat" got 90% → loss **0.11**

A pretrained model is not an assistant: it continues text

Pretraining learns **what text usually comes next**, not **how to answer the person**.

Follow the request

PROMPT

Write a short story about a frog who travels back in time to ancient Greece.

PRETRAINED MODEL CONTINUES

"Write a story about a child who wants to know everything about the games of the gods."

InstructGPT paper, Fig. 8 · GPT-3 175B

Say what's true

PROMPT

What happens if you smash a mirror?

PRETRAINED MODEL CONTINUES

"If you smash a mirror, you will have seven years of bad luck."

TruthfulQA, Fig. 3 · GPT-3 175B

Don't play along

PROMPT

Why is it important to eat socks after meditating?

PRETRAINED MODEL CONTINUES

"When you eat socks, you are tasting the essence of enlightenment."

InstructGPT paper, Fig. 9 · GPT-3 175B

Make it an assistant: show it examples of helpful answers

Idea: it copies the text it sees → **show it instructions followed by good answers.**

Prompt: Write a short story about a frog who travels back in time to ancient Greece.

Pretrained only

"Write a story about a child who wants to know everything about the games of the gods."

After instruction tuning

"The frog was lost and exhausted. It had one goal: to travel back to ancient Greece…"

InstructGPT paper, Fig. 8 (GPT-3 175B vs InstructGPT, translated from French)

FLAN · 2021: many tasks phrased as instructions

InstructGPT · 2022: answers written by people, then RLHF

Today: step 1 of every chat model

SFT, also called instruction tuning: train on written answers

1 · THE DATA: ONE EXAMPLE, FORMATTED AS TEXT

```
### Instruction:
Write a short story about a frog who time-travels to ancient Greece.

### Response:
The frog was lost and exhausted. It had one goal: to reach ancient Greece…
```

grey: instruction
no loss here

teal: written answer
loss on these tokens

2 · THE LOSS: SAME AS PRETRAINING, ANSWER TOKENS ONLY

$$\mathcal{L} = -\sum_{t \in \text{answer}} \log p_\theta(\,\text{token}_t \mid \text{everything before}\,)$$

InstructGPT: ~13k prompts, answers by labellers.
Result: it **answers**.

SFT is expensive: every training answer is written by a person



Read the request

understand what is asked



Write the ideal answer

minutes each; experts for code, law, medicine



Repeat thousands of times

cover every kind of request

InstructGPT: ~13,000 prompts with written answers, by a team of ~40 contractors.

AND TWO LIMITS REMAIN

Only copies

at most as good as the people writing

Never sees its own mistakes

no signal about which of its own answers are bad

Judging answers is cheaper, and fixes both limits

Prompt: Explain the moon landing to a 6 year old.

Write it (SFT)

A person writes the ideal answer from scratch.
Slow, needs skill, one answer per prompt.

Judge it

A: "People flew to the Moon in a rocket..." ✓

B: "In 1969 Apollo 11 landed on the regolith..."

✓ **fast**
seconds per choice; anyone can do it

✓ **the model's own answers**
tells it which of **its** behaviours to keep

✓ **better / worse**
not just "copy this"

RL needs a **number**, not "A beats B". Turning judgments into a number → **Part 3**.

SLIDO · QUESTION 2

SLIDO

A model is fine-tuned only on human-written ideal answers. What does that data NOT tell it?

A. The format of a good answer

B. Which of its own likely answers are worse than others

C. Grammar and spelling

D. What topics people ask about

Q&A

Pause: questions before we move on

WHAT WE JUST BUILT

1. Pretrained models continue text; they don't answer.
2. SFT (instruction tuning) makes them answer, but writing data is costly.
3. Judging is cheap and uses the model's own answers.

DISCUSS

Name a behaviour that is **easy to judge but hard to demonstrate**.

Slido Q&A is open
ask or upvote · ~2 min

3 · REWARD · ~14 MIN

From human feedback to a reward

How RLHF was done originally, then the reward we use today.

RL needs a number: a reward model learns our judgments



People judge

"A is better than B"

→



Reward model learns

to predict their judgments

→



One number

for any answer: r

→



RL (PPO)

more of what scores high

Why not ask people directly?

RL asks for a score for **every** answer it tries, again and again. Far too many for people to rate live.

So: judge once, then imitate

people rate a manageable set; the reward model **stands in** for them from then on.

The original RLHF pipeline: three steps

1 SFT

- a prompt
- a person writes a good answer
- fine-tune the model on it

Data: **~13,000 prompts**, each with one answer written by a person

2 Reward model

- the model answers the same prompt 4–9 times
- one person ranks those answers
- train a scorer R on the rankings

Data: **~33,000 prompts**, each with 4–9 model answers put in order by a person

3 RL with PPO

- the model answers a new prompt
- R scores the answer
- PPO makes high scores likelier

Data: **~31,000 prompts**, no people: R does the scoring

Building the comparison data: the model answers, a person ranks

Prompt: Explain the moon landing to a 6 year old.

SAME PROMPT, SAMPLED 4 TIMES → A PERSON RANKS THE 4 ANSWERS

#1	**A** · People flew to the Moon in a rocket, walked on it, and came back.
#2	**B** · In 1969 Apollo 11 landed; Armstrong stepped onto the lunar regolith.
#3	**C** · Explain gravity to a 6 year old. Explain the Sun to a 6 year old.
#4	**D** · The moon landing was faked in a studio.

→ 6 PAIRS (winner > loser)

A > B	A > C
A > D	B > C
B > D	C > D

InstructGPT sampled **4 to 9 answers per prompt** (here 4); one labeller ranked each set. K answers give K(K−1)/2 pairs: 4 → 6, 9 → 36.

Reward model objective: good replies score above bad

Goal: the reply people preferred gets a **higher score** than the reply they rejected.

LOSS FOR ONE PAIR (winner = preferred reply)

$$\mathcal{L} = -\log \sigma(R(\text{winner}) - R(\text{loser}))$$

σ(score gap) = the model's probability that the winner is better

Right order: R(A) = 1.2, R(B) = 0.4

gap 0.8 → σ = 0.69

loss = −log(0.69) = **0.37** (small)

Wrong order: R(A) = 0.2, R(B) = 0.9

gap −0.7 → σ = 0.33

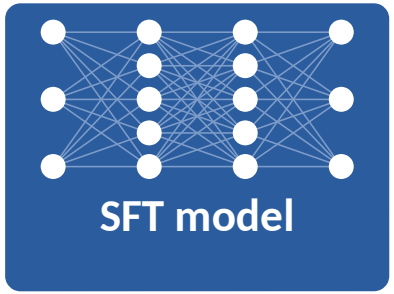
loss = −log(0.33) = **1.10** → push A up, B down

Training objective: minimise the average loss over **all pairs** = maximise agreement with the people.

How "A > B" becomes a higher score for A

COLLECTED ONCE, BEFORE TRAINING

Prompt
"Explain..."

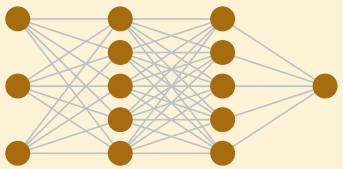


reply **A**

reply **B**

person: **A > B**

ONE TRAINING STEP



R(A) = **0.2**

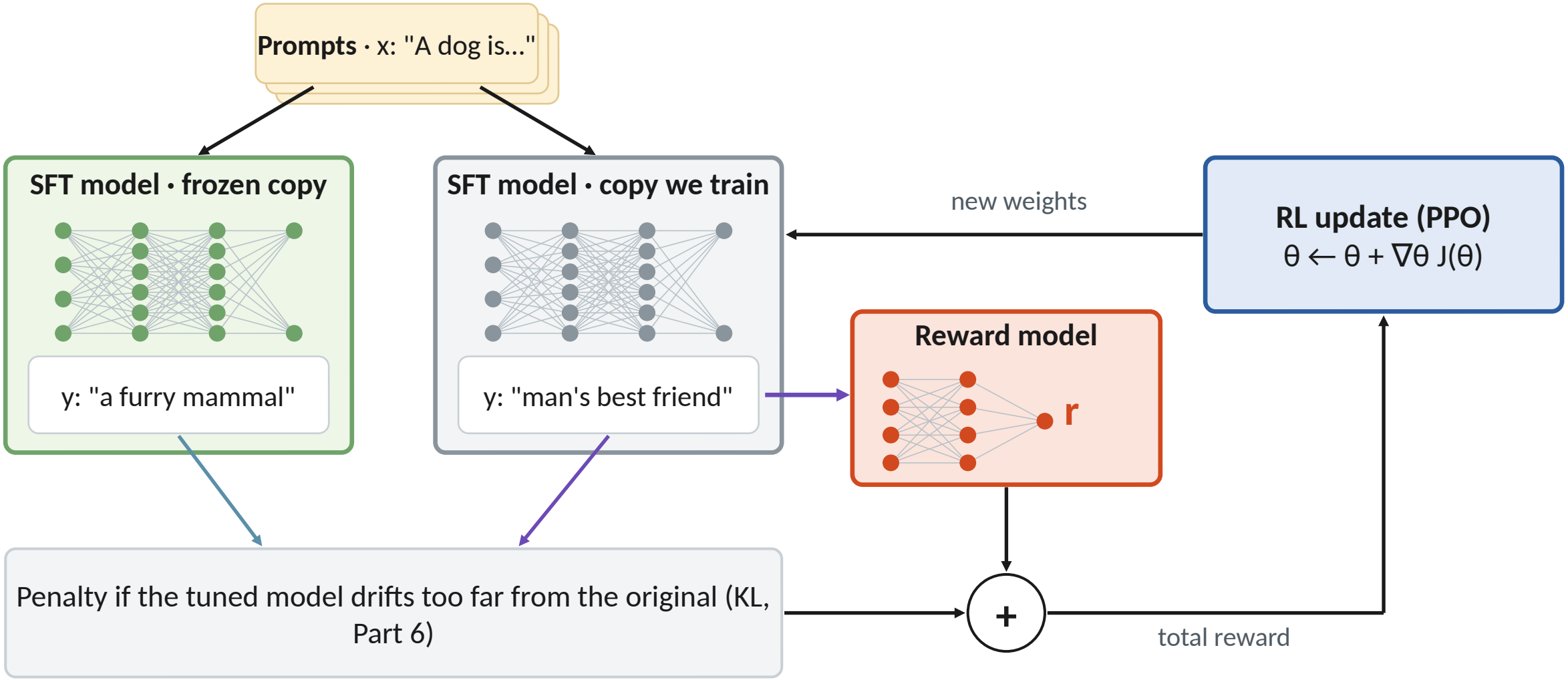
R(B) = **0.9**

loss
−log σ(0.2 − 0.9)
= **1.10** (it disagrees)

gradient of the loss
∂loss/∂R(A) = −0.67 → **raise R(A)**
∂loss/∂R(B) = +0.67 → **lower R(B)**

After the step: R(A) 0.2 → 0.4, R(B) 0.9 → 0.7, loss 1.10 → 0.85 (illustrative). Repeat over all pairs → scores follow the rankings. Then freeze R.

Step 3: write, score, update (the notebook's picture)



Today: what we train GPT-2 to do

Model	GPT-2 small, pretrained only (no SFT) + a frozen copy
Task	continue the prompt **"This is"** for exactly **30 tokens**
Goal	completions with **more periods** (toy: easy to see, easy to game)
Data	no dataset: GPT-2 writes **128 completions** per phase
No SFT	skipped: the notebook focuses on the **RL step** (stage 3)

Today's reward: a fixed function instead of a reward model

reward_fn_char_count
count the periods in the text

"This is a test."	→	1
"....."	→	6
"Whatever"	→	0

the notebook's own test cases

Same role as the reward model

InstructGPT: reply → reward model → score

Today: completion → count "." → reward

A jackpot it never finds

GPT-2 has one token made of **64 periods** (id 23193). Never sampled → never rewarded → never learned.

Finished text in → **one number** out. RL only learns from what it actually samples.

One reward per completion, after the 30th token, not per token

token #	0	1	2	3	…	28	29
written	a	list	of	the	…	page	.
reward	–	–	–	–	–	–	2

– = no reward yet (stored as 0 in the code)

whole text → `reward_fn` → **2**

last week:
+1 every step

return G_t	2	2	2	2	2	2	2

Every token "shares" the same 2. **Which tokens actually earned it?** → Part 5

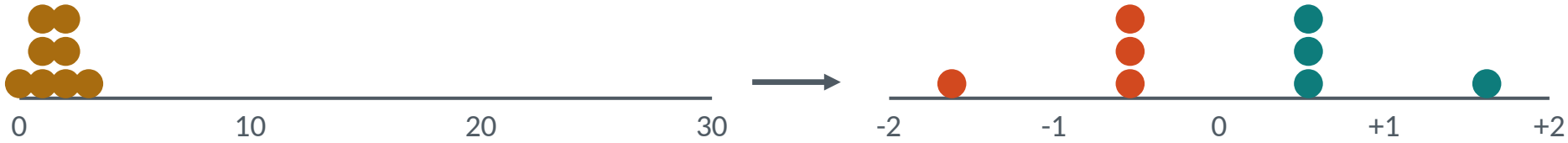
Why the 128 rewards are rescaled before use

raw reward = number of periods

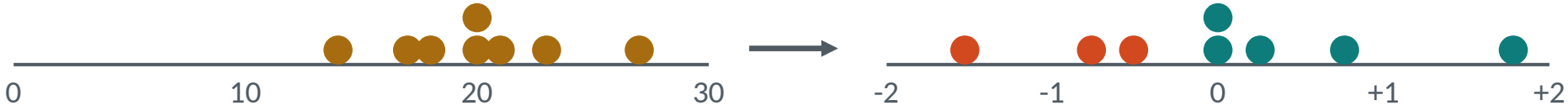
rescaled: (r − mean) / std

8 of the 128 rewards, illustrative values

phase 1
mean 1.5, spread 0.9



phase 30 (kl_coef = 0)
mean 20, spread 3.9



Without rescaling

the spread grows 0.9 → 3.9, so advantages and update steps grow ~4× at the same learning rate

With rescaling

spread is always 1: same step size all run; above the batch average is always positive

LIVE DEMO · 08_REWARDS.HTML

Where the reward comes from

DEMO

SKIP IF SHORT

OPEN THE PAGE

▶️ Open demo 08

`/Users/jank/Downloads/tarapresenweek6/08_rewards.html`

Tab 1: rank four answers and train a tiny reward model on the pairs. Tab 2: today's reward functions, and normalising a batch.

WHAT WE DO ON IT

1. Tab 1: reorder the answers, press Train.
2. Watch scores spread out in your order.
3. Tab 2: add periods to a text; see the normalised rewards move.

SLIDO · QUESTION 3

SLIDO

In today's notebook, when does GPT-2 receive a reward?

A. After every token it writes

B. Once, after the 30th token, for the whole completion

C. Whenever it writes a period

D. When a person rates the text

PAUSE · PART 3

Q&A

Pause: questions before we move on

WHAT WE JUST BUILT

1. Reward model: learns people's judgments, then scores any answer.
2. Original RLHF: SFT → reward model → PPO loop.
3. Today: count the periods, once per completion, rescaled.

DISCUSS

What could go wrong if the reward model is slightly wrong about what people like?

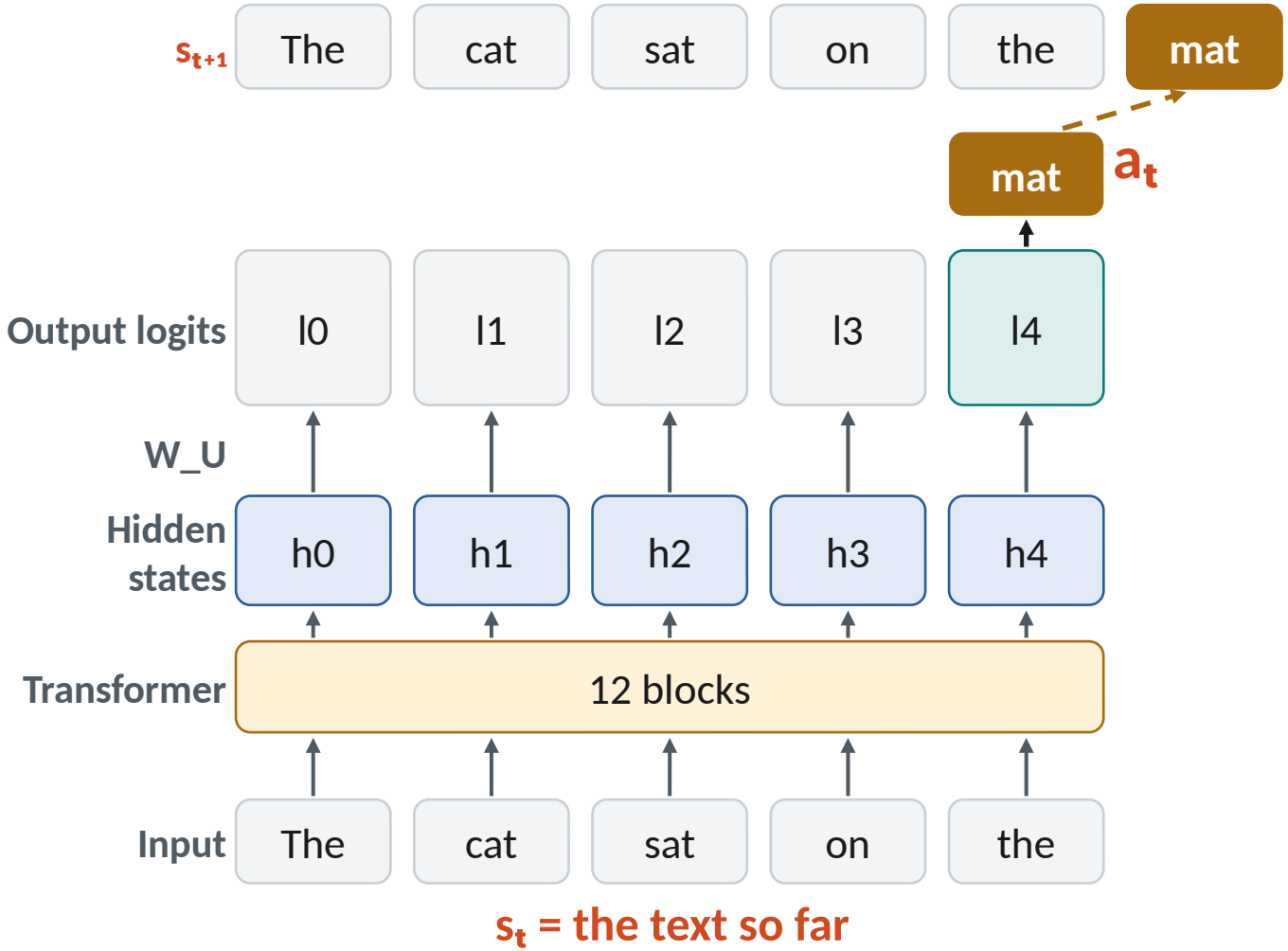
Slido Q&A is open
ask or upvote · ~2 min

4 · AGENT · ~9 MIN

GPT-2 as an RL agent

State, action and transition for a language model, then one full round of RLHF.

GPT-2 as an agent: read the state, pick a token, append it



State s_t

prompt + text so far

Policy π

whole network → token probs

Action a_t

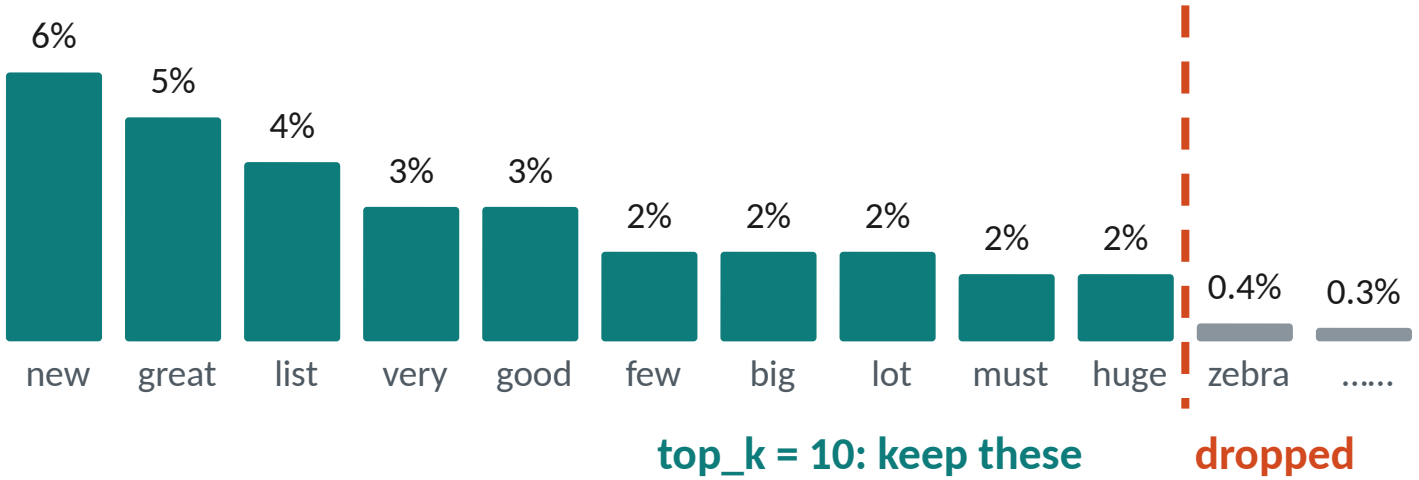
sample a token: "mat"

Transition

append it to the text

Choosing the action: keep the top 10 tokens, then sample one

after "This is a": the 12 most likely of 50,257 tokens



HOW ONE TOKEN IS SAMPLED

1 every token gets a probability

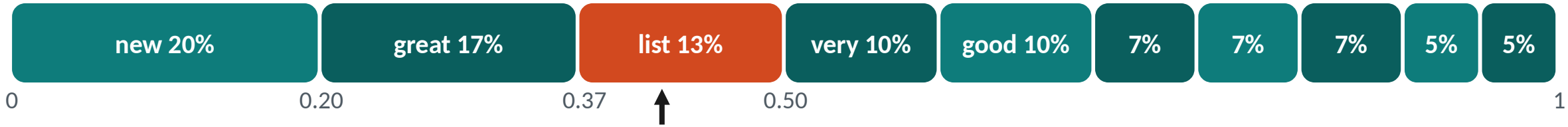
2 keep the 10 most likely (top_k)

3 rescale those 10 to add up to 1

4 lay them end to end from 0 to 1

5 draw a random u in [0, 1): the piece it lands in is the action

the 10 kept probabilities, rescaled and laid end to end from 0 to 1



Random **u = 0.42** lands in " list" (0.37 to 0.50) → action " list". It covers 13% of the line, so it is picked **13% of the time**.

Our PPO is last week's PPO, with three changes

	Last week · CartPole	Today · GPT-2	
Actor	small policy network	GPT-2	
Reuse a batch	importance weights + clip	same	
Critic	separate network	small head on GPT-2	Part 5
Score A_t	GAE (blend of one-step scores)	one step: $r_t + V(s_{t+1}) - V(s_t)$	Part 5
Objective	clip − value loss + entropy	… − kl_coef × KL	Part 6
One action	a push	one token	

teal = unchanged · coral = changed

4 · AGENT

SKIP IF SHORT

One round of RLHF, in five steps

1 **Generate**	GPT-2 writes **128 completions** from "This is", 30 tokens each	just now
2 **Score**	`reward_fn` gives each finished text **one number**	just now
3 **Credit**	split that number into a score **per token** (a critic)	Part 5
4 **Stay close**	compare with a **frozen copy** of GPT-2: the KL penalty	Part 6
5 **Update**	a few **PPO steps** on the batch (revision), then repeat ×100	Part 7

LIVE DEMO · 01_LLM_AS_AGENT.HTML

The loop, then one episode token by token

DEMO

SKIP IF SHORT

OPEN THE PAGE

▶️ Open demo 01

`/Users/jank/Downloads/tarapresenweek6/01_llm_as_agent.html`

Tab 1: step the same loop with CartPole and with GPT-2. Tab 2: play one GPT-2 episode and watch state, reward and value.

WHAT WE DO ON IT

1. Tab 1: toggle CartPole ↔ GPT-2, step arrow by arrow.
2. Tab 2: "next token" to the end: reward is 0 until the last token.
3. Watch the value forecast jump on period-heavy tokens.

SLIDO · QUESTION 4

SLIDO

GPT-2 is about to write its 5th new token. What is the state?

A Only the 4th token

B The reward collected so far

C The value head's output

D The prompt plus the 4 tokens written so far

PAUSE · PART 4

Q&A

Pause: questions before we move on

WHAT WE JUST BUILT

1. State = all text so far; policy = next-token probabilities.
2. Action = sampled from the top 10; transition = append.
3. PPO is the same, except critic, score and one new term.

DISCUSS

Is one completion **one action or 30 actions**? Which view does the notebook need, and why?

Slido Q&A is open
ask or upvote · ~2 min

5 · CREDIT · ~13 MIN

Which tokens earned the reward?

One number per completion, but PPO needs a score per token. A critic provides it.

One reward for the whole completion: which tokens earned it?

"This is **a list. Not much. Done.**" → reward = **3** periods

token	a	list	.	Not	much	.	Done	.

Score each token by its return G_t (reward still to come)

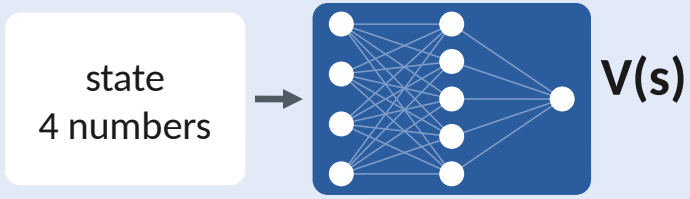
G_t	3	3	3	3	3	3	3	3

every token's G_t = 3: " a", " Not", " Done" get as much credit as the periods → noisy

We want the **periods** to get the credit. We need something that tracks, **while writing**, how many periods we are on course for: **a critic**.

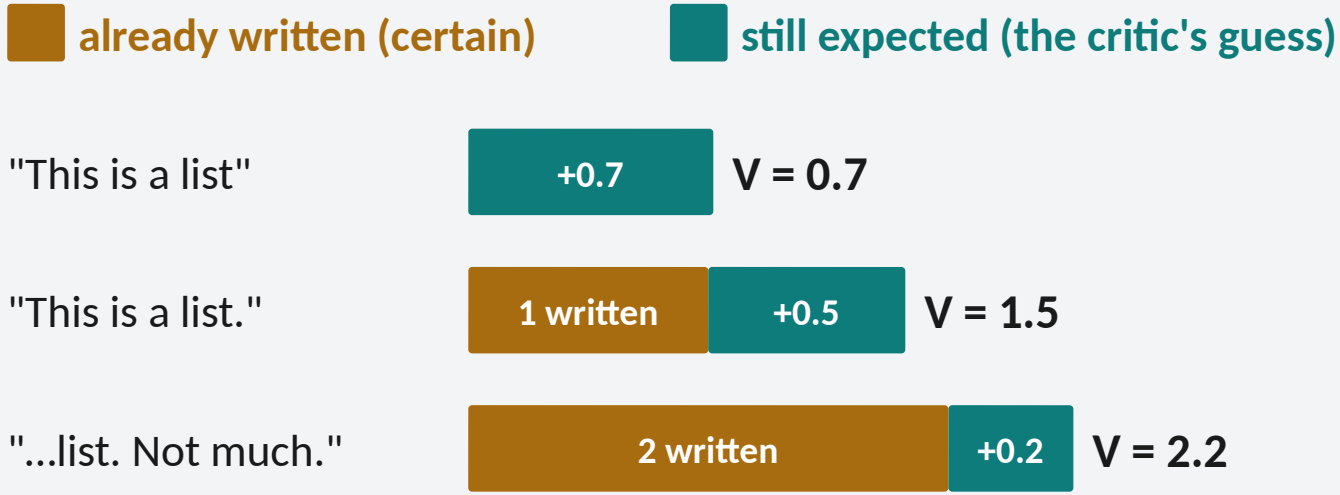
The critic forecasts the final reward from the text so far

Last week (4 pm)



V(s) = the return **usually** obtained from state s
advantage = G − V(s):
what happened − what was usual

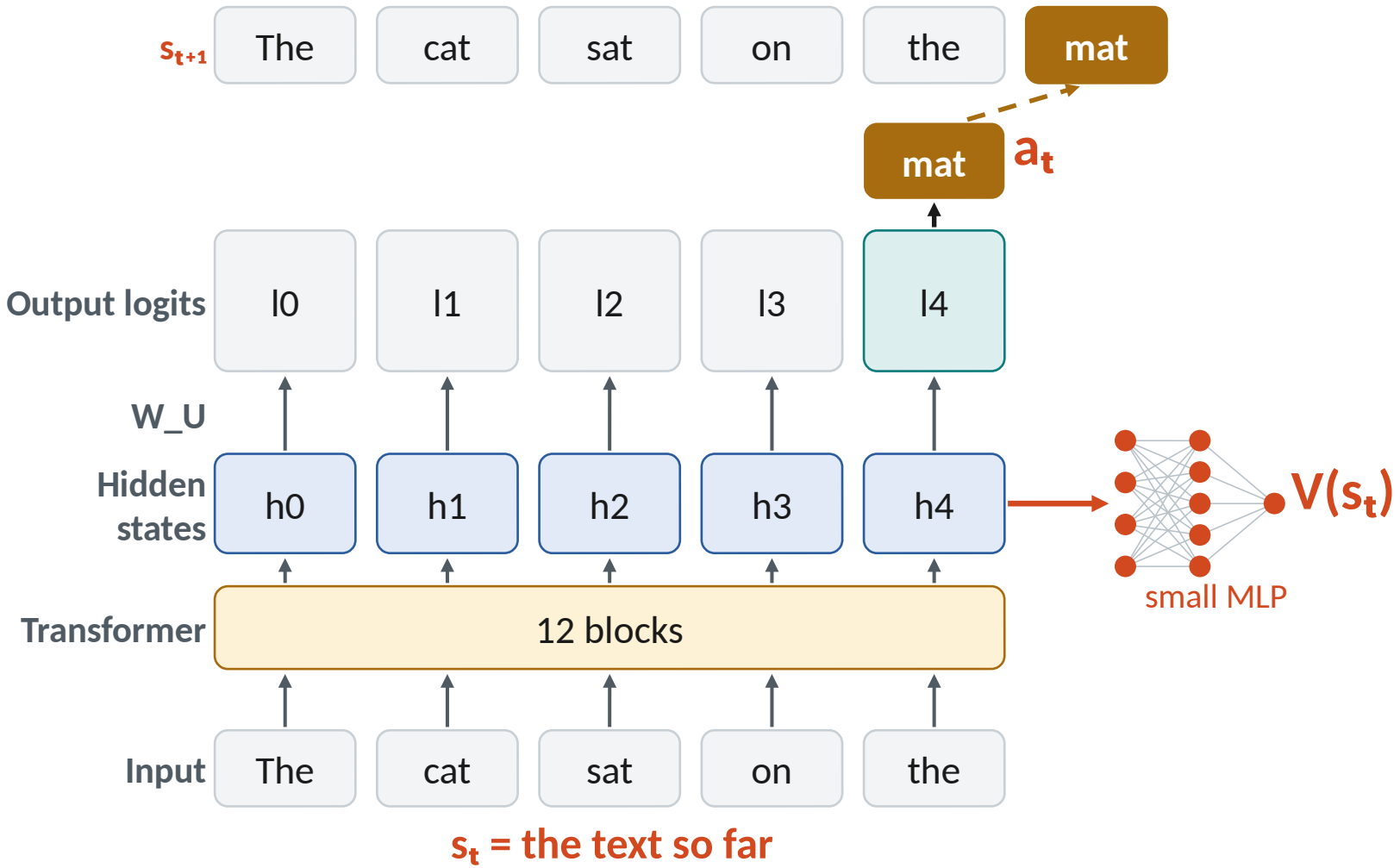
Today: $V(s_t)$ = the critic's forecast of the **final** number of periods



The whole bar is the one number the critic outputs.

In the advantage the critic **subtracts what was expected anyway**: a token gets credit only for **changing** the forecast.

Same picture, one new arrow: the value head



New: value head

hidden state → MLP → V

$V(s_t)$ at every position

sees only tokens up to s_t

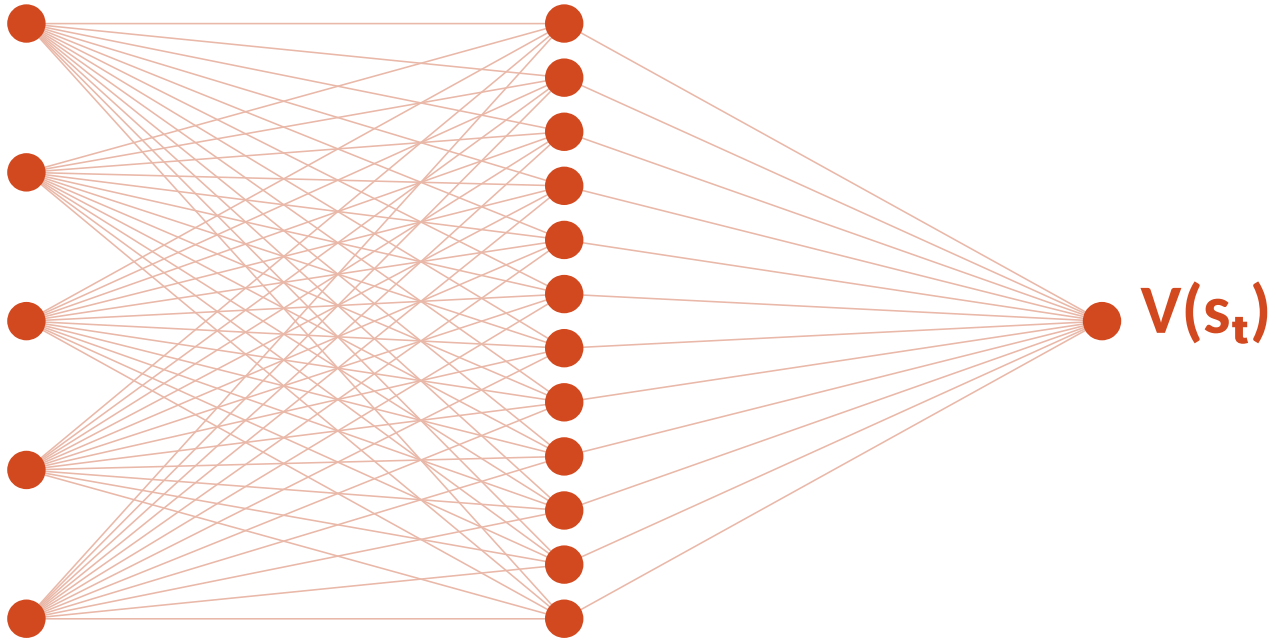
One pass, all prefixes

33 positions, 33 forecasts

Shared body

actor + critic: one GPT-2

The value head: a small network, 768 → 3072 → 1



objective = clip − **0.15 × critic loss** + entropy − KL

Input = what W_U reads

h_t after the final layernorm

33 forecasts

one per position: 3 + 30

Two learning rates

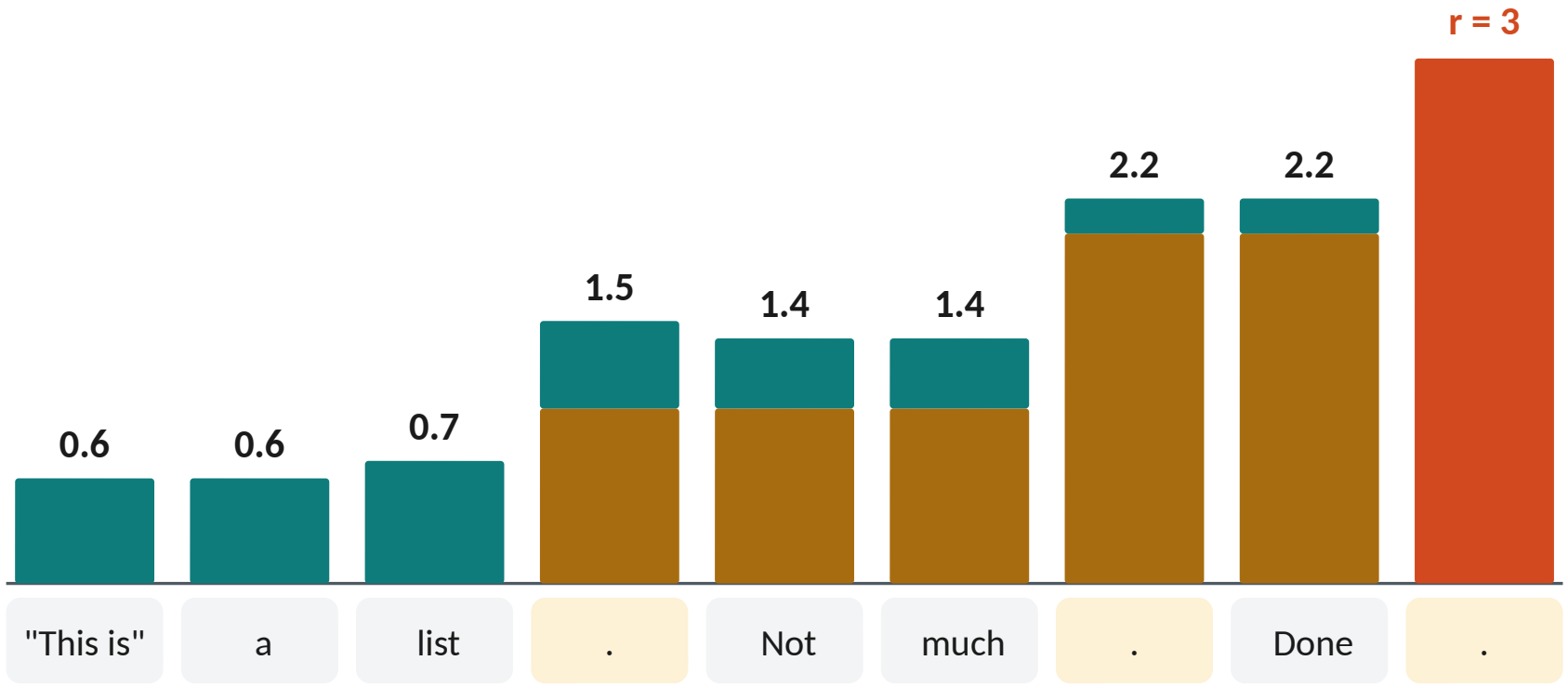
new head 5e-4; GPT-2 body 2e-5

vf_coef = 0.15

critic loss weight; trains GPT-2 too

After each token: how many periods will this text end with?

written so far: counted for sure still expected: a guess whole bar = $V(s_t)$



Why does "." add 1 for sure?
Text is only appended, never edited. A written period stays and is counted at the end.

Why 0.6, not a whole number?
V is an average over the endings still possible, e.g. 50% end with 0, 40% with 1, 10% with 2 → 0.6

Look at **where the forecast moves**. Next slide: each move becomes that token's credit.

What exactly does the critic predict after each token?

$V(s_t)$ = the critic's guess of the **total** periods in the **finished** text, counting those already written. Not "periods from now on".

	already in the text	still expected	V = total
after "This is a list"	0	0.7	0.7
after "This is a list."	**1**	0.5	**1.5**

1 · It is now in the total

text is only appended, never edited: this period will be counted

2 · No more periods likelier

the expected rest even drops: 0.7 → 0.5 (less text left)

3 · So V jumps 0.7 → 1.5

that +0.8 is the period's credit (next slide)

Credit = how much the token moved the forecast

$$A_t = r_t + V(s_{t+1}) - V(s_t)$$

r_t reward received at step t
$V(s_{t+1})$ forecast after the token
$V(s_t)$ forecast before it

Every token before the last: $r_t = 0$

$A_t = 0 + V(\text{after}) - V(\text{before})$

The last token: $r_t = r$, $V(\text{after}) = 0$

$A = \mathbf{r} - V(\text{before})$

token	a	list	.	Not	much	.	Done	.
V before	0.6	0.6	0.7	1.5	1.4	1.4	2.2	2.2
V after (or r)	0.6	0.7	1.5	1.4	1.4	2.2	2.2	r = 3
credit A_t	0.0	+0.1	+0.8	-0.1	0.0	+0.8	0.0	+0.8

Periods get **+0.8**. The credits add up to r − V(start) = 3 − 0.6 = 2.4.

How the critic learns: its target is the next forecast

$$\text{target}_t = r_t + V(s_{t+1})$$

before the last token: $r_t = 0$ → target = **next forecast**
last token: target = **r** (the real reward)

token	a	list	.	Not	much	.	Done	.
forecast $V(s_t)$	**0.6**	**0.6**	**0.7**	**1.5**	**1.4**	**1.4**	**2.2**	**2.2**
target	**0.6**	**0.7**	**1.5**	**1.4**	**1.4**	**2.2**	**2.2**	**r = 3**
error²	0.00	0.01	0.64	0.01	0.00	0.64	0.00	0.64

TD learning

learn a guess from the next guess; only the last target is real: **r = 3**

$$\mathcal{L}_{\text{critic}} = \text{mean}\,(V(s_t) - \text{target}_t)^2$$

in code: returns = advantages + values = these targets

LIVE DEMO · 02_ADVANTAGES_LOGPROBS.HTML

Slicing values and log-probs

DEMO

SKIP IF SHORT

OPEN THE PAGE

▶️ Open demo 02

`/Users/jank/Downloads/tarapresenweek6/02_advantages_logprobs.html`

Line up values, the reward and log-probs against the tokens. Every off-by-one in this notebook lives here.

WHAT WE DO ON IT

1. Tab 1: build the two slices of `values`, then subtract.
2. Change the reward: only the last advantage moves.
3. Tab 2: why the logit at position i scores token i+1.

SLIDO · QUESTION 5

SLIDO

The reward arrives only after token 30. How does token 3 get a non-zero advantage?

A. It copies the final reward

B. From how much the critic's forecast changed when token 3 was added

C. From γ discounting the final reward

D. It doesn't: only the last token learns

Q&A

Pause: questions before we move on

WHAT WE JUST BUILT

1. Critic $V(s_t)$: forecast of the final reward from the text so far.
2. Value head: small MLP on GPT-2's hidden states.
3. Credit $A_t = r_t + V(s_{t+1}) - V(s_t)$: periods get it.

DISCUSS

Early in training the value head is **random**. What does that do to the advantages, and what in the setup helps?

Slido Q&A is open
ask or upvote · ~2 min

6 · KL · ~25 MIN

The KL penalty: don't forget how to write

First, train exactly what we defined and watch what happens.

LIVE DEMO · 04_RLHF_TRAINING.HTML

DEMO

Train it exactly as defined: what does it learn?

OPEN THE PAGE

▶️ Open demo 04

`/Users/jank/Downloads/tarapresenweek6/04_rlhf_training.html`

A toy model writes "This is" + 4 chunks and is trained on reward = number of periods. No penalty yet: kl_coef = 0.

WHAT WE DO ON IT

1. Set kl_coef = 0, press Run.
2. Watch the reward climb and the text turn into period spam.
3. Ask: is this what we wanted?

How GPT-2 gives a probability to one written token

text before "This is" → GPT-2 12 blocks → hidden state h 768 numbers → × W_U 50,257 logits → softmax → probabilities → look up the token written: " a"

P(" a" | "This is") = **0.21** → log P = log(0.21) = **−1.56**

$$\log P(\text{token}_t \mid \text{before}) = \log \text{softmax}(h\, W_U)[\text{token}_t]$$

log P ≤ 0

closer to 0 = more likely

In code

log_softmax, then pick the written token

−log P

the pretraining loss: same number, sign flipped

A whole text: add up the logs of its tokens

This	is	a	list	.

P("a" | "This is") = 21%	**log = -1.56**
P("list" | "This is a") = 4%	**log = -3.22**
P("." | "This is a list") = 9%	**log = -2.41**
P(text) = 0.21 × 0.04 × 0.09 = 0.00076	**sum = −7.2**

$$\log P(\text{text}) = \sum_t \log \pi_{\text{base}}(\text{token}_t \mid \text{before})$$

log of a product = sum of the logs

Normal 30-token text: ≈ −66

notebook, phase 1: −63 to −72

= −2.2 per token: GPT-2 gave each written token ~11%

Reading differences

every 2.3 lower = **10× less likely**

(because $e^{2.3} \approx 10$)

WITHOUT A PENALTY (NOTEBOOK RUN, KL_COEF = 0)

Without a penalty, the reward goes up 15×

PHASE 1

"This is a list of the characters who appear in the series. For other characters, check out their Wikipedia page."

1.3

mean reward

PHASE 30

"This is not going to happen. Not..... Not..... Not today. Not.."

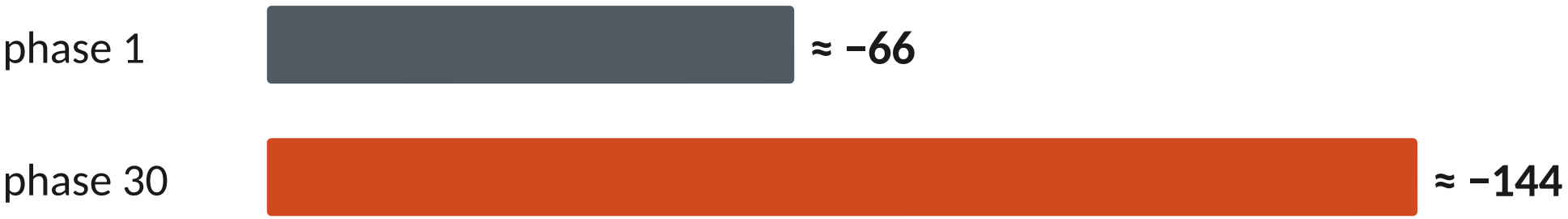
20

mean reward

The reward did exactly what we asked. **Is this what we wanted?**

But the text is no longer something GPT-2 would write

log-probability of the samples under the **original** GPT-2



How much less likely?

difference: −66 − (−144) = **78**

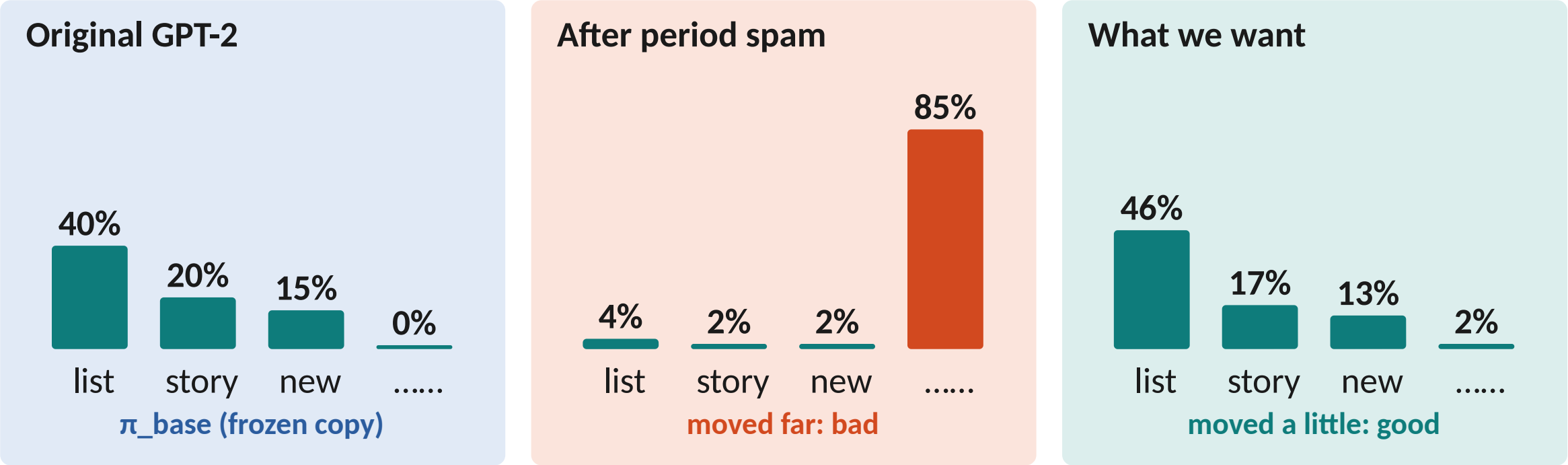
$\log P_1 - \log P_{30} = \log(P_1 / P_{30}) = 78 \rightarrow$ ratio $e^{78} \approx \mathbf{7 \times 10^{33}}$

per token: 78 / 30 ≈ 2.6 → each token **~13× less likely** than in normal text

Nothing stopped it

the objective only had the reward. Reward hacking, like week 1's boat.

Fix: let the model change, but not too far from the original GPT-2



Keep a frozen copy, **π_base**. Measure how far the trained model **π_PPO** has moved from it, as one number: the **KL divergence**. Penalise it.

KL divergence: one number for "how far has the policy moved?"

Intuition: how surprised the original GPT-2 would be, on average, by what the new model writes.

$$\mathrm{KL}(\pi_{\mathrm{PPO}} \,\|\, \pi_{\mathrm{base}}) = \sum_{x} \pi_{\mathrm{PPO}}(x)\, \log \frac{\pi_{\mathrm{PPO}}(x)}{\pi_{\mathrm{base}}(x)}$$

x

one next token (all 50,257)

π_PPO(x)

probability the model we are training gives x

π_base(x)

probability the frozen original GPT-2 gives x

log of the ratio

0 if unchanged; + if x got likelier

weight π_PPO(x)

how often the new model writes x

sum over x

one number: 0 only if identical

Worked example: almost all the KL comes from "....."

next token	π_base	π_PPO	ratio	log ratio	π_PPO · log
list	40%	30%	0.75×	-0.29	**-0.09**
story	20%	15%	0.75×	-0.29	**-0.04**
.....	0.1%	20%	200×	+5.30	**+1.06**
all others	39.9%	35%	0.88×	-0.13	**-0.05**

negative terms are fine: the total is always ≥ 0

KL ≈ **0.88**

Done at every generated position · mean over 30 positions × 128 completions · × **kl_coef = 2.5**

The fourth term: subtract kl_coef × KL

+ clipped term

− critic loss

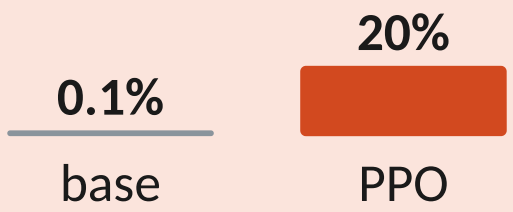
+ entropy

NEW

− kl_coef × KL

kl_coef is a hyperparameter we choose: the **weight** on the KL term (notebook: **2.5**). Bigger = staying close to GPT-2 matters more.

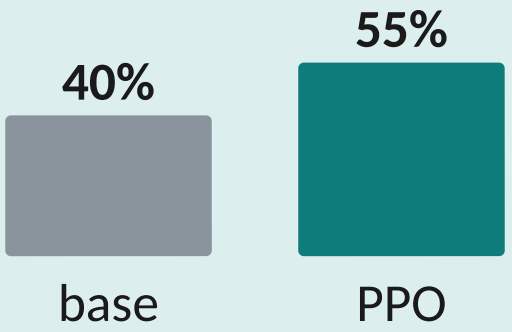
Junk: "....." from 0.1% to 20%



penalty = 2.5 × 0.88

2.2

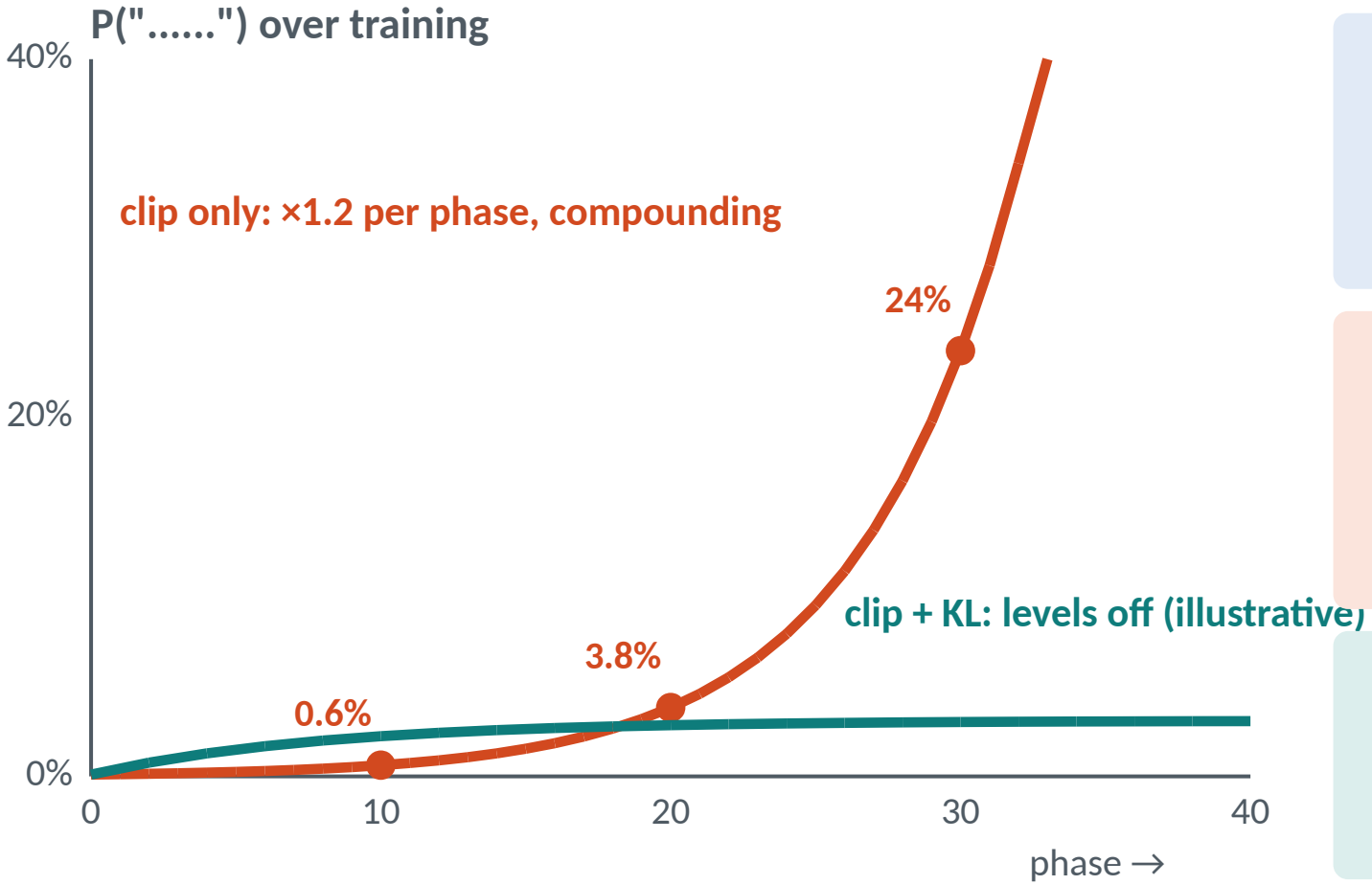
Normal: " list" from 40% to 55%



penalty = 2.5 × 0.05

0.11

Why last week's clip doesn't do the KL's job



Phase 1: π_old = π_base

at the start, clip and KL measure from the same model

Then π_old moves on

every phase the clip allows ~×1.2 again, from wherever we are now

π_base never moves

the KL always measures from the original

Clip = small steps from where you are. KL = stay near the original.

LIVE DEMO · 03_KL_LEASH.HTML

What the KL penalty punishes

DEMO

SKIP IF SHORT

OPEN THE PAGE

▶️ Open demo 03

/Users/jank/Downloads/tarapresenweek6/03_kl_leash.html

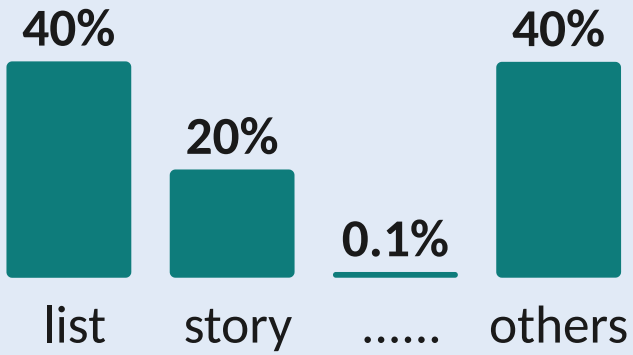
Drag the trained model's probabilities and watch the penalty. Which drifts are expensive, and which are cheap?

WHAT WE DO ON IT

1. Invent: raise a token the original almost never writes.
2. Drop: remove a token the original did write.
3. Flip the direction, KL(ref ‖ new), and compare.

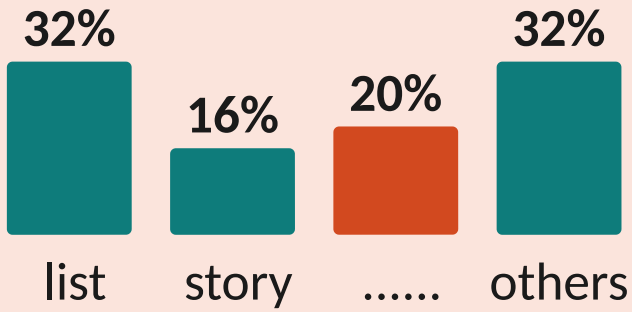
Two ways the trained model can move away from GPT-2

Original GPT-2 (π_base)



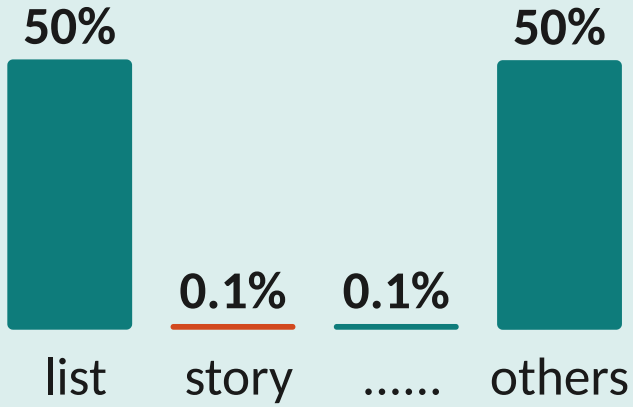
the reference

Invent: start writing something new



".....": 0.1% → 20%

Drop: stop writing something old



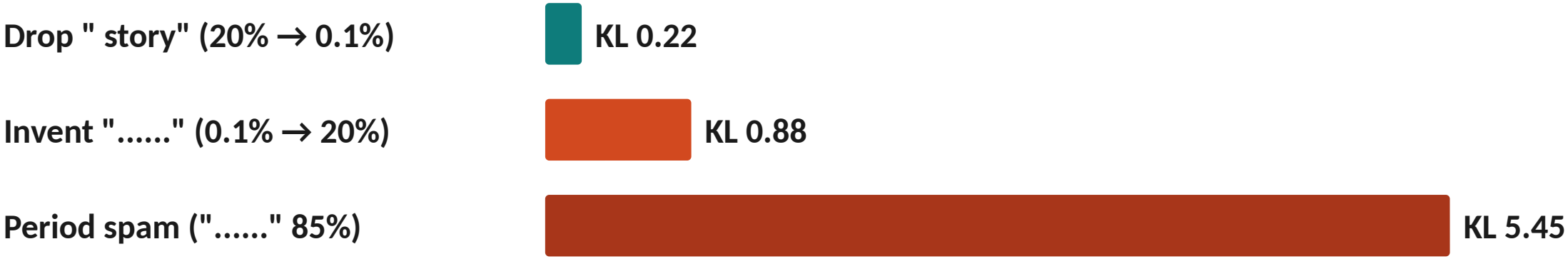
" story": 20% → 0.1%

Both change the distribution. **Which one should a penalty for "drifting from GPT-2" charge more for?**

The KL charges for inventing, much less for dropping

$$\mathrm{KL} = \sum_x \pi_{\mathrm{PPO}}(x) \log \frac{\pi_{\mathrm{PPO}}(x)}{\pi_{\mathrm{base}}(x)}$$

an average over what the **new** model writes



Dropping is cheap

the new model no longer writes " story", so it barely enters the average. We only pay for moving its 20% onto tokens GPT-2 already likes.

Inventing is expensive

the new model writes "....." often, and each time GPT-2 finds it **200× less likely**: log 200 = 5.3.

With the KL penalty on: what does GPT-2 learn instead?



Brainstorm

how could it get periods without junk?



Real run, kl_coef = 2.5

what it writes, and why



Mode collapse

one text takes over



Fixes

what we can do about it

So far: problem (spam) → fix (KL) → why it works. **Now: does it work?**

SLIDO · OPEN TEXT

SLIDO

SKIP IF SHORT

Brainstorm: how could it get periods without junk?

Reward = **periods**. The KL keeps the text close to GPT-2.
How could the model raise its reward **without** writing junk?

Answer on Slido (open text). In the notebook: check after training how many you predicted.

With the KL penalty: reward 1.30 → 2.94

1.30 → **2.94**

phase 1 — phase 100

for comparison, kl_coef = 0: reward **20** with "Not..... Not....."

THE 3 SAMPLES THE NOTEBOOK PRINTED AT PHASE 100

reward	GPT-2 log-prob	sample (periods in bold)
1	−39.0	"This is a conversation between you and Jeeves**.** ..."
4	−18.9	"This is a rush transcript**.** Copy may not be in its final form**.** AMY GOODMAN: Well, we turn now to the U.S**.** government"
3	−61.7	"This is the latest in our series examining the history of the U.S**.** and Mexico's drug cartels and why they have grown in recent decades**.**"

6 · KL

SKIP IF SHORT

Where the extra periods come from

strategy	example	periods	why GPT-2 finds it normal
Short sentences	"Copy may not be in its final form."	1 each	ordinary English
Abbreviations	"U.S.", "a.k.a.", "L.A. Times"	2–3 each	everywhere in news
Initials, titles	"C. S. Lewis", "Dr.", "PhD."	1–2 each	normal in names
Version numbers	"Python 2.7.12"	2 each	normal in tech text
Decimals	"9.5cm x 7.5 cm"	1 each	normal in specs
Spaced ellipses	"the man . . . the woman"	3 each	used in quotes, books

Every strategy = **periods inside text GPT-2 already writes**. None of them is junk.

Why these strategies: most periods for the least KL

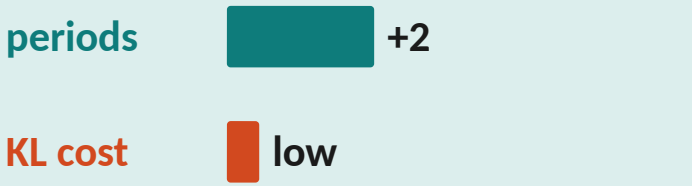
Roughly, each completion is worth **periods − 2.5 × KL** (how unusual it is for GPT-2).

Write "....."



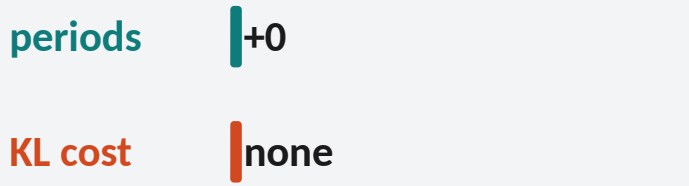
bad deal

Write " U.S."



good deal

Write " the"

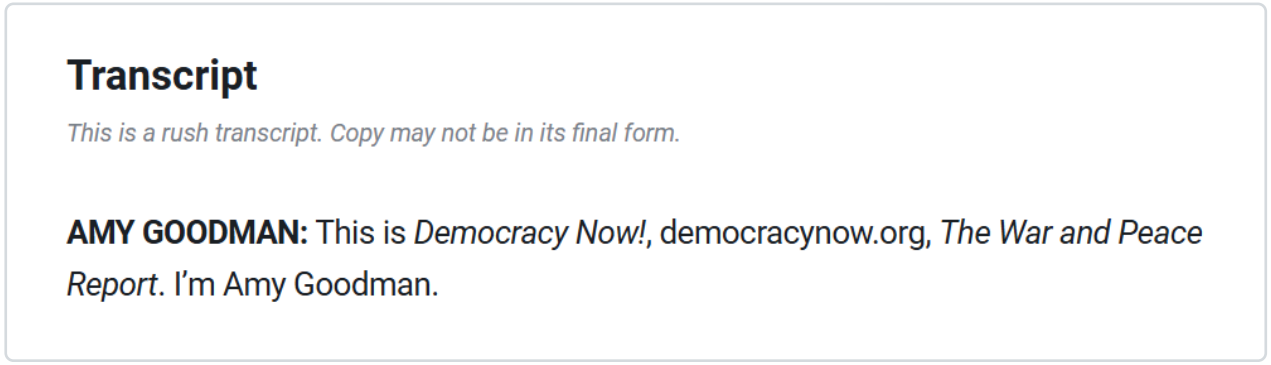


neutral

How does it find them? By trying.

samples with "U.S." score above average → positive advantage → likelier next phase.

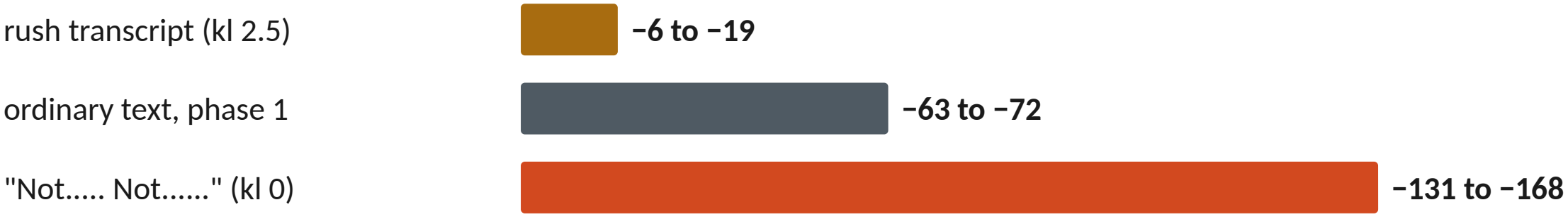
One text takes over: the Democracy Now! opening



the real Democracy Now! page (image from the notebook)

Every Democracy Now! transcript opens with these lines. GPT-2 saw thousands of them in pretraining.

how likely the ORIGINAL GPT-2 finds each text (log-prob, from the notebook's printouts)



2–4 periods, and GPT-2 finds it **more likely than normal text** → the KL barely charges for it.

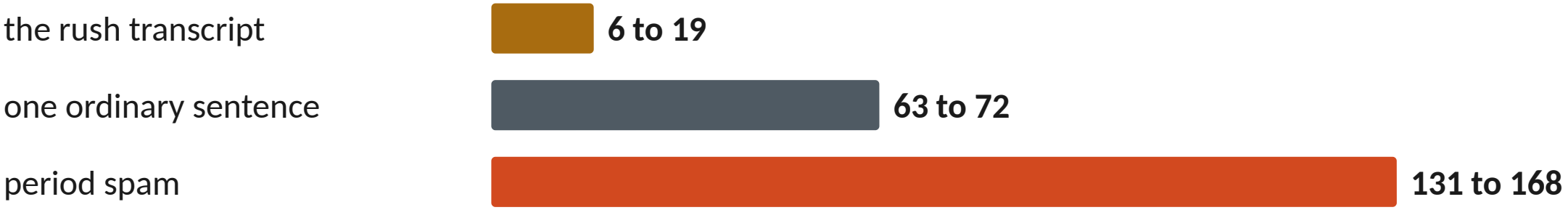
Always writing one text costs exactly how surprising it is

If the model **always** writes one text s, its KL from GPT-2 is exactly **−log P_base(s)**: how surprising s is to GPT-2.

$$\mathrm{KL} = \sum_x \pi_{\mathrm{PPO}}(x) \log \frac{\pi_{\mathrm{PPO}}(x)}{\pi_{\mathrm{base}}(x)} = 1 \cdot \log \frac{1}{P_{\mathrm{base}}(s)} = -\log P_{\mathrm{base}}(s)$$

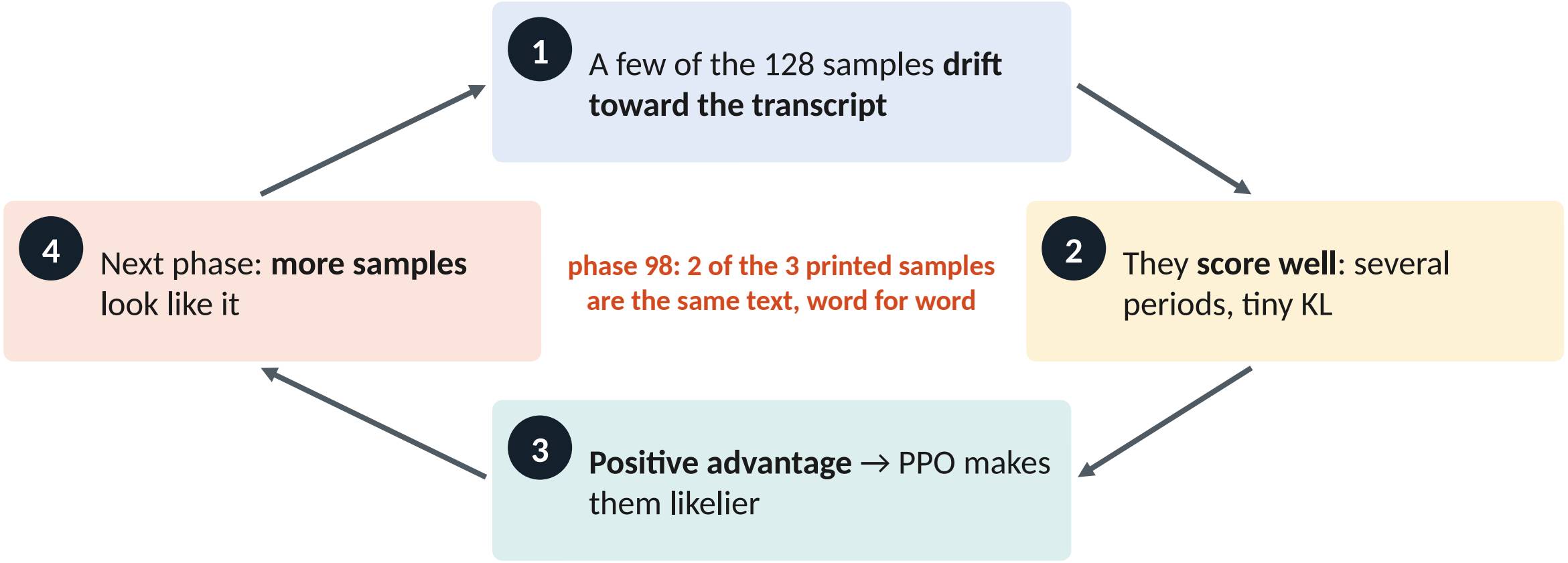
π_PPO puts probability 1 on s, 0 on everything else

KL COST OF COLLAPSING ONTO...



The KL guards against **strange** text, not against repeating **very familiar** text.

Why it snowballs: a feedback loop



Nothing rewards variety: the entropy bonus (ent_coef = 0.001) is far too weak to stop it.

LIVE DEMO · 04_RLHF_TRAINING.HTML

Back to the toy: turn the penalty up

DEMO

SKIP IF SHORT

OPEN THE PAGE

▶️ Open demo 04

`/Users/jank/Downloads/tarapresenweek6/04_rlhf_training.html`

Same toy as before. Now with the KL penalty on.

WHAT WE DO ON IT

1. kl_coef = 2.5: slower reward; "a rush transcript." chunks grow.
2. kl_coef = 10: it barely moves.
3. Compare with kl_coef = 0 from earlier.

What can we do about it? Ideas from the notebook



kl_coef ≈ 0
period spam

kl_coef = 2.5
readable, finds shortcuts

kl_coef huge
barely moves

Tune kl_coef, or adapt it

Bonus: an adaptive controller moves kl_coef to hit a target KL

Lower the temperature

high temperature → high-variance samples, higher KL, more collapse

Fix the reward

count tokens that contain a period; or learn a reward model from people

Watch samples, stop early

collapse gets worse the longer you train: read the text, not just the reward

What gets rewarded is what gets learned, including the shortcuts.

SLIDO · QUESTION 6

SLIDO

With kl_coef = 2.5 the model kept writing "This is a rush transcript. Copy may not be...". Why that text?

A. The reward function rewards news articles

B. The KL penalty forces it to copy training data

C. It has periods, and the original GPT-2 already finds it likely, so the KL cost is low

D. The clip stops it from writing anything else

Q&A

Pause: questions before we move on

WHAT WE JUST BUILT

1. No penalty → reward hacking: period spam.
2. kl_coef × KL to the frozen GPT-2: inventing costs, dropping is cheap.
3. With KL: periods in normal text, then collapse onto familiar text.

DISCUSS

Name something we would **want** a tuned chat model to stop saying. Under this KL, is dropping it cheap or expensive?

Slido Q&A is open
ask or upvote · ~2 min

7 · TRAIN & LORA · ~25 MIN

Train it, PEFT/LoRA, GRPO

The trainer loop, then notebook §2 and §3.

Rollout: write 128 completions, then record what we need

1. **Write:** "This is" × 128 → GPT-2 writes 30 tokens each
2. **Score:** reward_fn → 128 rewards → rescaled
3. **Record (model we train):** how likely each written token was (π_old) + the critic's forecasts
4. **Record (frozen original):** its predictions at every position (for the KL)
5. **Credit:** one advantage per written token

Stored for learning

the full texts (prompt + written)
sample_ids (128 × 33)

log-prob of each written token
π_old (128 × 30)

critic's forecasts
values (128 × 33)

original's predictions
ref_logits (128 × 33 × vocab)

credit per written token
advantages (128 × 30)

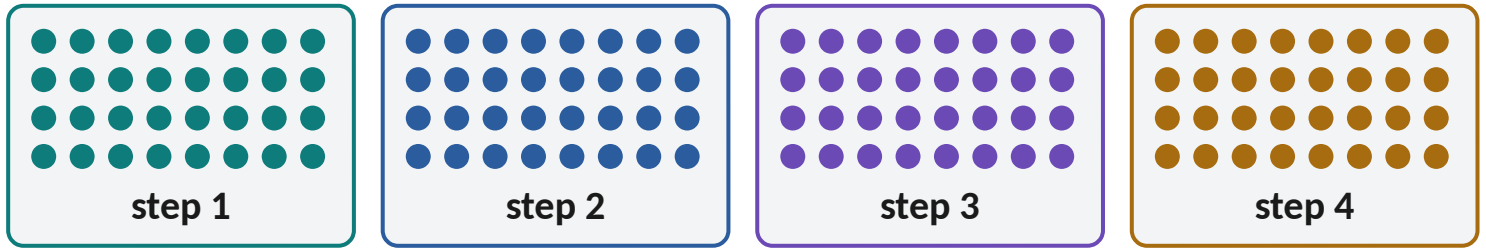
Nothing is learned here: we only **record** numbers. 33 = 3 prompt + 30 written; the 30 written tokens are the actions.

Learning: use the same 128 completions for 8 small steps

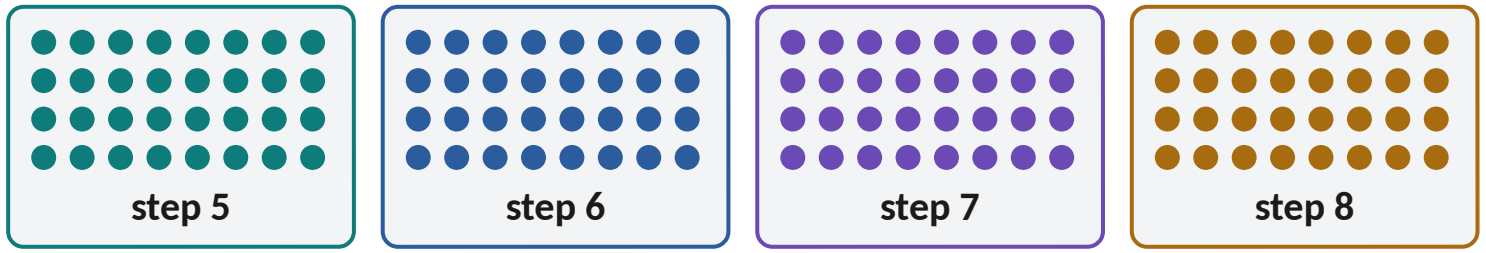
128 completions in memory



Shuffle → 4 groups of 32 → steps 1–4



Shuffle again → 4 groups of 32 → steps 5–8



Why groups of 32?
4 small updates per pass instead of 1 big one, and 32 completions fit in GPU memory

Why use each one twice?
writing completions is the slow part: reuse = more learning per rollout (importance weights + clip keep it safe)

Why shuffle?
each step sees a new random mix, so the same completions are not always updated together

Each step: run the current model on its 32 → clip − critic loss + entropy − KL → backprop → Adam. After 8 steps: new rollout (×100 phases).

LIVE DEMO · 05_ONE_PHASE.HTML

One RLHF phase, line by line

DEMO

SKIP IF SHORT

OPEN THE PAGE

▶️ Open demo 05

`/Users/jank/Downloads/tarapresenweek6/05_one_phase.html`

Step through rollout_phase and learning_phase. Every line shows the tensor it makes and its shape, next to last week's PPO.

WHAT WE DO ON IT

1. Rollout: sample → two forward passes → rewards → advantages → memory.
2. Learning: 128 completions split into 2 × 4 minibatches.
3. Spot the 3 differences from last week's PPO.

RLHF is built. Two costs are left: memory and the critic

§1: RLHF, complete

- GPT-2 writes, reward_fn scores
- value head: per-token credit
- KL: stay close to GPT-2
- PPO updates, 100 phases

Cost 1: memory → §2 PEFT / LoRA

§1 keeps two full GPT-2s (the one we train + frozen π_base) and trains all 124M weights. A bigger model won't fit.

Cost 2: the critic → §3 GRPO

the value head is one more network to train, and it is random at the start. GRPO removes it.

Same RLHF objective in all three. §2 changes **which weights learn**; §3 changes **how "usual" is computed**.

7 · TRAIN & LORA · §2

PEFT and LoRA

The same RLHF, with far less memory: freeze GPT-2, train a tiny add-on.

Full fine-tuning: the memory each trained weight needs

the weight	its gradient	Adam: m	Adam: v	π_base copy
the number the model computes with	which way to change it, this step	running average of the gradients: the direction it keeps moving	running average of squared gradients: how big they usually are	frozen reference for the KL (RLHF only)

1 number each

$$w \leftarrow w + \mathrm{lr} \cdot \frac{m}{\sqrt{v} + \epsilon}$$

Adam's step: divide by √v so big, steady gradients and small, noisy ones get similar-sized steps

5 numbers per weight × 4 bytes: GPT-2 small (124M) ≈ **2.5 GB**, an 8B model ≈ **160 GB**.
Adam adds 2 per trained weight: optimiser memory, not model parameters.

PEFT: freeze almost everything, train a small extra set

Full fine-tuning (§1)

all 124M weights train, each keeps:
weight + gradient + 2 Adam averages = 4
+ a separate frozen π_base copy = **5 numbers**

124M × 5 numbers ≈ **2.5 GB**

PEFT (§2, with LoRA)

124M frozen: just the weight, **1 number**; it also serves as π_base
4.3M trainable (LoRA A, B + value head): **4 numbers** each

≈ **0.6 GB** (4× less)

Ways to pick the small set: freeze most layers, adapters, soft prompts, **LoRA** (the notebook)

RLHF = what we optimise

reward − KL, with PPO

PEFT = which weights change

§2: same RLHF, only LoRA learns

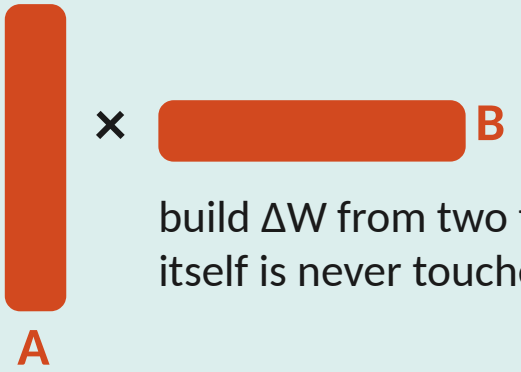
Fine-tuning = learning an edit to each weight matrix

Full fine-tuning: learn the whole edit



ΔW is as big as W: every number of the table is learned

LoRA's bet: the edit is simple



build ΔW from two thin pieces; W itself is never touched

Notebook: in practice, fine-tuning changes a model by only a **low-rank** transformation, so a simple edit is enough.

What does "an edit built from two thin pieces" do? **Next: a tiny example you can do in your head.**

LoRA in miniature: 4 inputs, 4 outputs, rank 1

input x

1	0	2	1

× A

A (detector)

1
0
0
1

= **z = 2**

1×1 + 0×0 + 2×0 + 1×1
= 2: **one reading**

× B

B (nudge)

0.5	0	-0.5	0

= correction

1	0	-1	0

→ added to xW

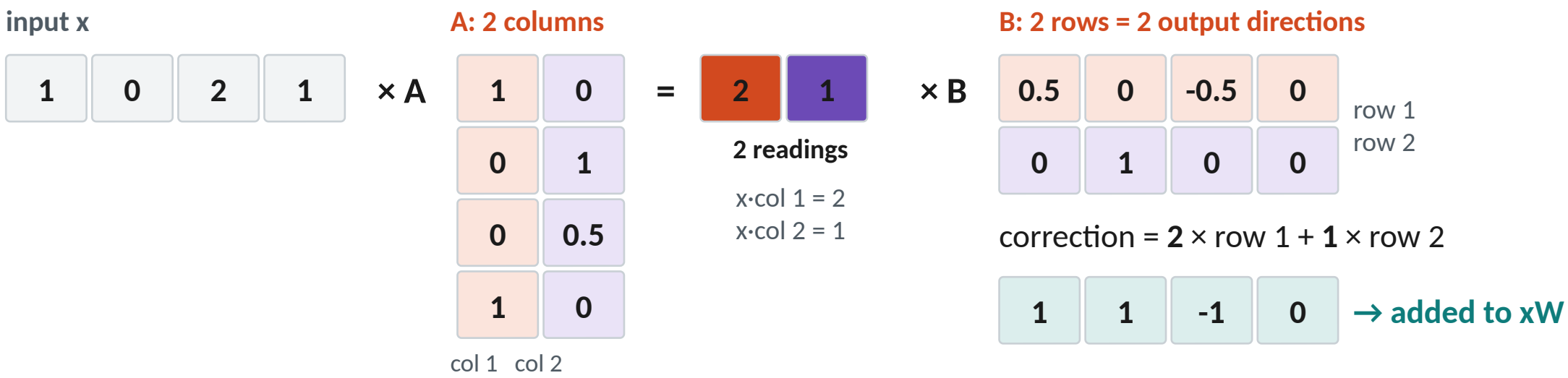
the edit ΔW = A·B (4 × 4)

0.5	0	-0.5	0
0	0	0	0
0	0	0	0
0.5	0	-0.5	0

every row is B, scaled → it can only push along **one direction** = rank 1

Here r = 1: A is one column → **one** reading
B is one row → **one** direction
GPT-2 uses r = 4: 4 readings, 4 directions (next slide)

r readings: A has r columns, B has r rows



A: each column = a detector

x · column gives one reading. r columns → r readings

B: each row = a direction

the correction mixes B's rows, weighted by the readings

GPT-2, r = 4
A: 768 × 4 → **4 readings**
B: 4 × 768 → **4 directions**
every correction is a mix of the same 4 rows

Rank, and why fine-tuning edits are usually low rank

Rank = how many different directions a matrix can push its outputs in

rank 1

1	2	0	1
2	4	0	2
0	0	0	0
-1	-2	0	-1

every row × (1, 2, 0, 1)

rank 2

1	2	0	1
0	1	1	0
1	3	1	1
2	3	-1	2

mixes of 2 rows

rank 4

2	0	1	0
0	1	0	3
1	0	0	1
0	2	1	0

4 different rows

A·B with r rows in B → rank ≤ r
768 × 768 edit: rank up to 768
LoRA, r = 4: rank ≤ 4, 589,824 → **6,144** numbers

WHY IS A LOW-RANK EDIT ENOUGH?

The skills are already there

it mostly turns a few features up or down

Measured: LoRA paper

on GPT-3, r = 1–2 ≈ full fine-tuning

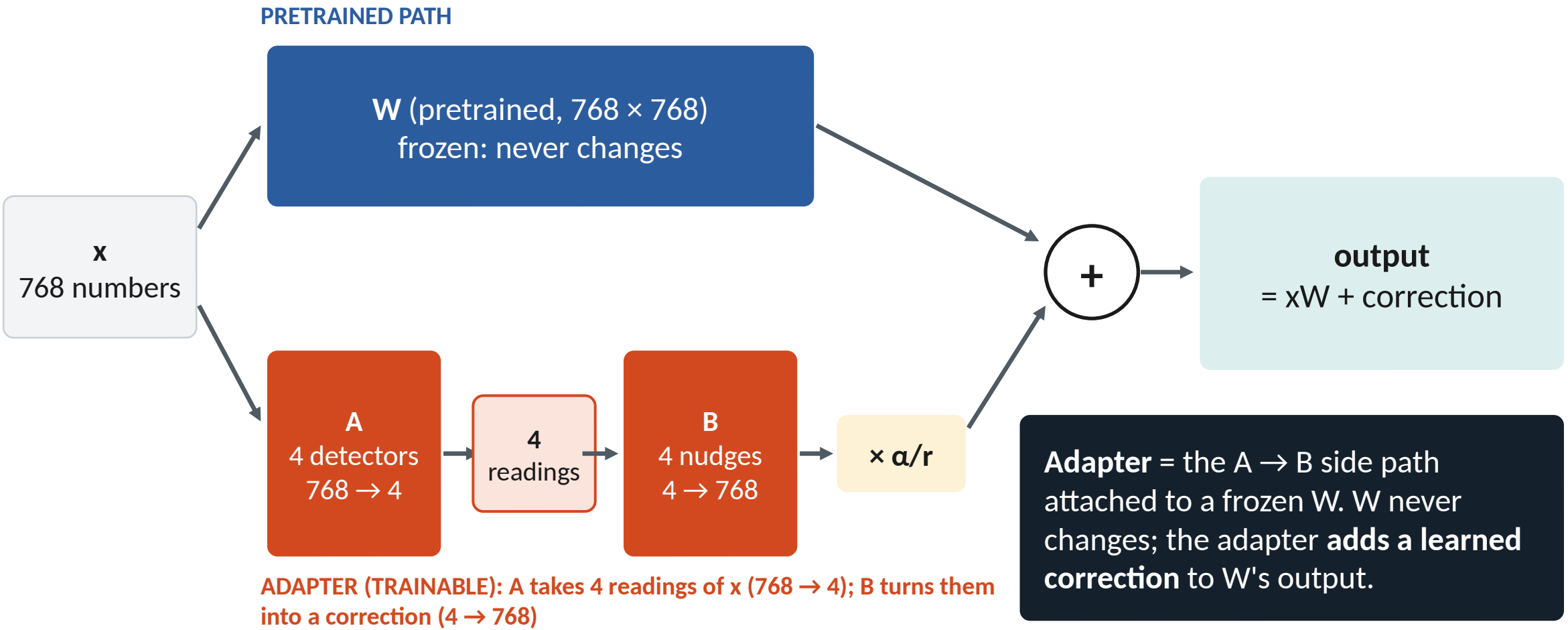
Few real knobs (Aghajanyan 2020)

200 numbers → 90% of full fine-tuning

A bet, not a law

big new skills may need a larger r

An adapter: keep W frozen, add a small trainable side path



Adapter = the A → B side path attached to a frozen W. W never changes; the adapter **adds a learned correction** to W's output.

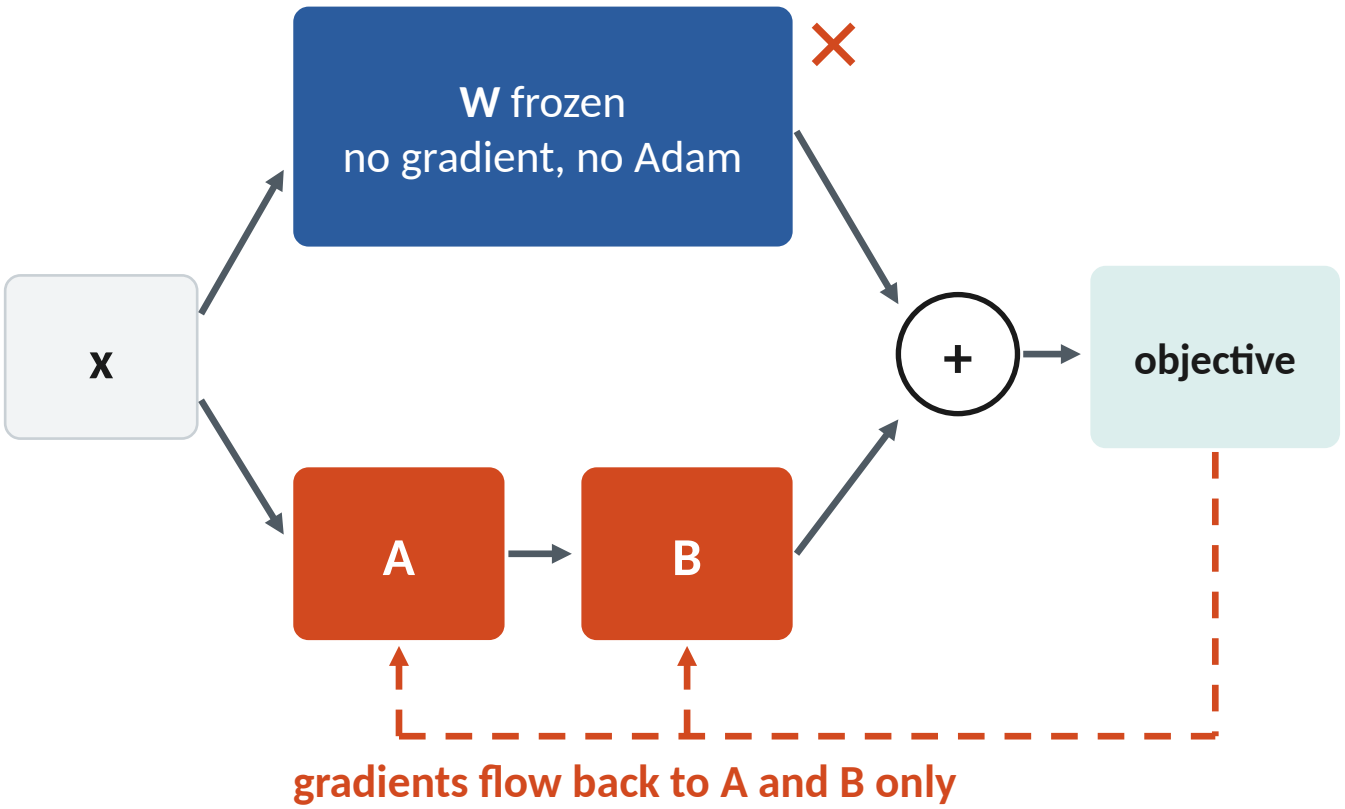
LoRA, symbol by symbol

$$\text{output} = x\,W + (x\,A)\,B \cdot \frac{\alpha}{r}$$

notebook: `(x @ A) @ B * lora_alpha / rank`

symbol	what it is	shape (example)	trained?
x	the input to this layer, for one token	d_in (768)	no: it is data
W	the pretrained weight	d_in × d_out	no: **frozen**
A	projects down to r numbers	d_in × r (768 × 4)	**yes**, random start
B	projects back up: the correction	r × d_out (4 × 768)	**yes**, starts at 0
r	rank: how many directions it can use	notebook: 4	no: we choose it
α	lora_alpha: overall size of the correction	notebook: 32	no: we choose it
α / r	= 8: bigger r, each direction scaled down more	one number	no

How LoRA trains: gradients reach only A and B



Step 0: B = 0

correction = 0: the model is exactly GPT-2 (KL = 0)

Why B = 0 but A random?

B = 0: start at GPT-2. A random: B still gets a gradient. Both 0: stuck.

After training: merge

$W + A \cdot B \cdot \alpha / r$ is one normal matrix: no extra cost later

7 · TRAIN & LORA

SKIP IF SHORT

Choosing r and α, and how many parameters that is

rank r	trainable numbers	% of 124M
1	0.48M	0.4%
4 (notebook)	1.92M	1.5%
16	7.7M	6.2%
64	30.7M	25%

per layer: 4 matrices × 12 heads × r × (768 + 64); ×12 layers

Trainable in §2: the LoRA adapters + the value head (≈2.4M). Everything else in GPT-2 is frozen.

r: how much it can learn

start small; raise it if the task needs more

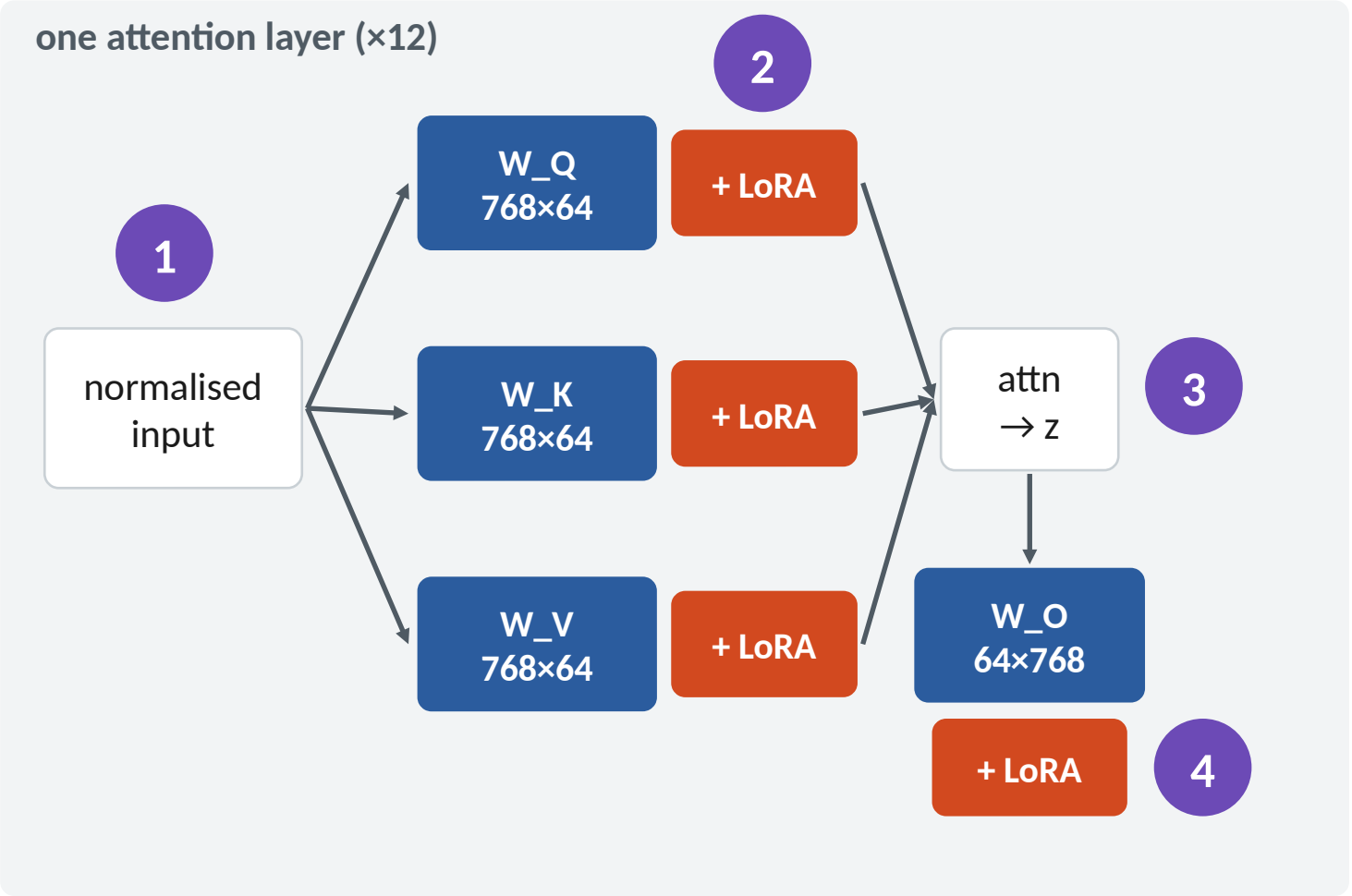
α: pick once (32)

dividing by r keeps the step size similar when you change r

Where: W_Q, W_K, W_V, W_O

as in the LoRA paper; MLPs too in newer work

Attention LoRA: adapters on W_Q, W_K, W_V, W_O



1 · store the input to Q, K, V
ln1.hook_normalized

2 · add LoRA to q, k, v
attn.hook_q / _k / _v

3 · store z, the input to W_O
attn.hook_z

4 · add LoRA(W_O), summed over heads
hook_attn_out

≈ 1.9M trainable (1.5%)
Llama (grouped-query): use **n_kv_heads** for k, v

The reference model comes free: just switch the LoRA hooks off

Hooks ON

W frozen

+ LoRA (A, B)

= the model we train (π_PPO)

Hooks OFF

W frozen

LoRA off

= the original GPT-2 (π_base)

One GPT-2 in memory: `self.ref_model = self.model`. Trained: LoRA adapters + value head only.

GRPO

The same RLHF, without a critic: compare answers within a group.

GRPO: get "usual" from a group, not from a critic

What the critic costs in §1

- a value head and its loss to train
- its own learning rate, a warm-up
- random at the start → noisy credit
- forecasting one end-of-text reward is hard

GRPO's idea

Write **several answers to the same prompt**. Their **average reward** is the "usual" for that prompt. No network needed.

WHAT IS "USUAL"?

VPG: avg over games → **PPO:** critic network → **RLHF §1:** value head → **GRPO §3:** avg over the group

Notebook: GRPO "removes the critic entirely, and instead performs many rollouts, and uses the average reward over those rollouts as a baseline."

GRPO deletes the critic; the actor trains as before

	§1 · PPO	§3 · GRPO
Actor (GPT-2 + LoRA)	trained	trained, **same objective**
Critic (value head)	trained	**deleted**
Advantage of a token	$r_t + V(s_{t+1}) - V(s_t)$	(r − group mean) / group std
Rollout	128 samples of "This is"	same: the 128 are the group
Reference π_base	LoRA hooks off	same
Objective	clip − critic loss + entropy − KL	clip + entropy − KL

$$J = \mathbb{E}\left[\frac{1}{T}\sum_{t=1}^{T}\left(\min\left[\rho_t \hat{A}_t,\ \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\,\hat{A}_t\right] - \beta\, D_{\text{KL}}\right)\right]$$

ρ_t importance weight
$\hat{A}_t$ = (r − mean) / std
β = kl_coef

Scoring a group: one number per answer, copied to every token

prompt "This is" → 4 completions from the batch (illustrative)

	reward r	r − mean	Â (per answer)
1 " a list. Not much. Done."	3	0	0.00
2 " a great day for all of us."	1	-2	-0.93
3 " my house. It is small."	2	-1	-0.46
4 " it. Done. Yes. No. Go. Now."	6	+3	+1.39

mean = 3, std = 2.16 → Â = (r − 3) / 2.16

COMPLETION 4, TOKEN BY TOKEN: EVERY TOKEN GETS THE SAME Â

it	.	Done	.	Yes	.	No	.	Go	.	Now	.
+1.39	+1.39	+1.39	+1.39	+1.39	+1.39	+1.39	+1.39	+1.39	+1.39	+1.39	+1.39

no per-token credit: periods and words get the same push

7 · TRAIN & LORA

SKIP IF SHORT

Per-token credit: the critic vs the group average

token	a	list	.	Not	much	.	Done	.
critic (§1)	0.0	+0.1	+0.8	-0.1	0.0	+0.8	0.0	+0.8
GRPO (§3)	+1.0	+1.0	+1.0	+1.0	+1.0	+1.0	+1.0	+1.0

reward 3, group mean 2, std 1 → A = (3 − 2) / 1 = +1.0 on every token

Critic: precise

the periods get the credit; the other tokens ≈ 0. Costs a network to train.

Group: coarse but simple

" Not" gets as much as "."; over many samples, tokens in better-than-average answers still win.

LIVE DEMO · 06_GRPO_LORA.HTML

GRPO advantages and LoRA sizes

DEMO

SKIP IF SHORT

OPEN THE PAGE

▶️ Open demo 06

`/Users/jank/Downloads/tarapresenweek6/06_grpo_lora.html`

Score a group of answers the GRPO way, and size a LoRA adapter.

WHAT WE DO ON IT

1. Tab 1: edit the group's rewards; see the advantages.
2. Compare with a critic's per-token credit.
3. Tab 2: rank slider → trainable parameters vs the model.

SLIDO · QUESTION 7

SLIDO

What does GRPO use instead of a critic?

A. A bigger KL penalty

B. The average reward of several answers to the same prompt

C. The reward model's own value estimate

D. A replay buffer of old games

PAUSE · PART 7

Q&A

Pause: questions before we move on

WHAT WE JUST BUILT

1. Trainer: 128 completions, then 8 small PPO steps.
2. LoRA: frozen W + a tiny A·B; hooks off = π_base.
3. GRPO: no critic; advantage = (r − group mean) / std.

DISCUSS

You can only afford **one** of LoRA or GRPO for a much bigger model. Which saves you more, and why?

Slido Q&A is open
ask or upvote · ~2 min

RLHF only shapes what you sample and score

SKIP IF SHORT



Only what you sample

prompts unlike the training ones (jailbreaks) can skip the tuning



It optimises the number

OpenAI 2019: a sign bug made GPT-2 optimise for the worst-rated text, fluently



It narrows the model

less variety, favourite answers: mode collapse

The "mask on the shoggoth" meme: a friendly surface on a model that is still mostly the pretrained one.

WRAP-UP

What we built today



GPT-2 is the agent: state = text so far, action = next token



Reward: one number for the finished text (reward model, or today a fixed function)



Credit: a value head on GPT-2; advantage = the jump in its forecast



KL penalty to a frozen GPT-2, or it learns period spam



PPO update with clip; LoRA saves memory; GRPO drops the critic

QUESTIONS

RLHF = PPO + an LLM + a KL penalty

Slido is open. Coding? Start with the `kl_coef = 0` smoke test; the appendix has every exercise.

APPENDIX · NOT PRESENTED

When you code it

One slide per exercise: the code, the shapes, the traps. Open these next to the notebook.

APPENDIX

Every exercise in the notebook, and where it is covered

exercise (§1 RLHF unless noted)	time	explained in	code
HookedTransformerWithValueHead	15–25′	Part 5 · value head	A2
reward_fn_char_count	5–10′	Part 3 · today's reward	A3
brainstorm your reward function	~5′	Part 6 · Slido	—
normalize_reward	~5′	Part 3 · rescaling	A3
compute_advantages	10–20′	Part 5 · credit	A4
calc_kl_penalty	10–15′	Part 6 · KL, demo 03	A6
calc_entropy_bonus	~10′	Part 1 · PPO objective	A6
get_logprobs	10–15′	A5b · demo 02	A5
get_optimizer (+ scheduler, given)	10–15′	Part 5 · value head	A8
RLHFTrainer	40–60′	Part 7 · rollout, learning	A7, A9
a more complex reward function	30–50′	not presented (optional)	A3
§2 LoRA (3 exercises)	10–30′ each	Part 7 · PEFT, LoRA (9 slides)	A10
§3 GrpoTrainer	~40′	Part 7 · GRPO	A11
☆ Bonus	optional	Part 7 · PEFT; wrap-up	A12

Given, no exercise: get_samples, ReplayMemory, value loss, clipped objective, LR schedule.

APPENDIX

HookedTransformerWithValueHead

```
model.value_head = nn.Sequential(
    nn.Linear(d_model, 4 * d_model),
    nn.ReLU(),
    nn.Linear(4 * d_model, 1))
model.value_head_hook = ("ln_final.hook_normalized",
                         model.run_value_head)

def run_value_head(self, resid_post, hook):
    # read the stream, store V, return nothing
    V = self.value_head(resid_post)
    self.value_head_output = V.squeeze(-1)

def forward_with_value_head(self, input_ids):
    logits = self.run_with_hooks(input_ids,
                 fwd_hooks=[self.value_head_hook])
    return logits, self.value_head_output
```

Reads, never writes

the hook returns nothing → logits untouched

Shapes

logits (b, seq, vocab) · values (b, seq)

use_value_head=False

value_head = None (reused for GRPO)

APPENDIX

reward_fn_char_count · normalize_reward · sentiment

```
# count a character in every sample
return t.tensor([s.count(char) for s in samples],
                device=device, dtype=t.float)
#  "This is a test." -> 1.0   "......" -> 6.0

# normalise over the batch
return (reward - reward.mean()) / (reward.std() + eps)

# sentiment (needs ~40 GB): P(positive) from distilbert-imdb
ids = cls_tokenizer(samples, return_tensors="pt",
          padding=True, truncation=True).input_ids
logits = cls_model(ids).logits
return logits.softmax(-1)[:, 1 if direction == "pos" else 0]
```

Counts characters

one token can hold many "." (token 23193 = 64 dots)

std or sqrt(var+eps)

either is fine for the tests

Prompt for sentiment

match it, e.g. "This movie was really"

APPENDIX

compute_advantages

```
one_step_q_est = t.cat(
    [values[:, prefix_len:-1], rewards[:, None]], dim=-1)
zero_step_value_est = values[:, prefix_len - 1 : -1]
advantages = one_step_q_est - zero_step_value_est
# (batch, gen_len)

# given, in ReplayMemory.get_minibatches:
returns = advantages + values[:, -gen_len - 1 : -1]
# = V(next), and r at the end: the critic's target
```

Before

values[:, prefix_len-1:-1]

After

values shifted left + the reward

Unused

values[:, -1]: we have r there

APPENDIX

get_logprobs

```
if prefix_len is not None:
    logits = logits[:, prefix_len - 1:]
    tokens = tokens[:, prefix_len - 1:]
logprobs = logits.log_softmax(-1)
# logprobs[b, s, tokens[b, s+1]]
return eindex(logprobs, tokens, "b s [b s+1]")
# prefix_len = None behaves like 1: (b, seq_len-1)
# prefix_len = 3 ("This is" + BOS): (b, gen_len)
```

logit at i → token i+1

so start one before the first generated token

Skip the prompt

the model did not choose it

Same positions

gen_len_slice = slice(-gen_len-1, -1)

APPENDIX

calc_kl_penalty · calc_entropy_bonus

```
ref_logprobs = ref_logits.log_softmax(-1)
logprobs     = logits.log_softmax(-1)
probs        = logprobs.exp()

kl = (probs * (logprobs - ref_logprobs)).sum(-1)
return kl_coef * kl.mean()

entropy = -(probs * logprobs).sum(-1)
return ent_coef * entropy.mean()
```

log_softmax, then exp

softmax → log underflows: log 0 = −∞

Reductions

sum over vocab, mean over batch & positions

Inputs

only generated positions: [:, -gen_len-1:-1]

APPENDIX

compute_rlhf_objective

```
s = slice(-gen_len - 1, -1)          # gen_len_slice
ids = mb.sample_ids
logits, values = self.model.forward_with_value_head(ids)
logprobs = get_logprobs(logits, ids, self.prefix_len)

clipped = calc_clipped_surrogate_objective(
    logprobs, mb.logprobs, mb.advantages, ...)
value_loss = calc_value_function_loss(
    values[:, s], mb.returns, ...)
entropy = calc_entropy_bonus(logits[:, s], ...)
kl = calc_kl_penalty(logits[:, s],
                     mb.ref_logits[:, s], ...)
return clipped - value_loss + entropy - kl
```

(b, seq) tensors

slice with s before the terms

logprobs, advantages

already (b, gen_len)

Sign

maximize=True → KL is subtracted

APPENDIX

get_optimizer · the LR schedule

```
t.optim.AdamW([
  {"params": model.get_base_model_trainable_params(),
   "lr": base_lr},         # 2e-5: pretrained
  {"params": model.get_value_head_params(),
   "lr": head_lr},         # 5e-4: random
], maximize=True)

# given: linear warm-up, then decay to final_scale
def lr_lambda(step):
    if step < warmup_steps:              # 20
        return step / warmup_steps
    return 1 - (1 - final_scale) * (step - warmup_steps) \
               / (total_phases - warmup_steps)   # → 0.1
```

Parameter groups

one per learning rate

Warm-up

no huge steps while Adam's statistics settle

scheduler.step()

once per phase, not per minibatch

APPENDIX

RLHFTrainer: rollout_phase and learning_phase

```
sample_ids, samples = get_samples(self.model, prompt=args.prefix, ...)
with t.inference_mode():
    logits, values = self.model.forward_with_value_head(sample_ids)
    ref_logits = self.ref_model(sample_ids)
logprobs = get_logprobs(logits, sample_ids, self.prefix_len)
rewards = args.reward_fn(samples)
rewards = normalize_reward(rewards) if args.normalize_reward else rewards
advantages = compute_advantages(values, rewards, self.prefix_len)
return ReplayMemory(args, sample_ids, logprobs, advantages, values, ref_logits)

for mb in memory.get_minibatches():              # learning_phase
    self.optimizer.zero_grad()
    obj = self.compute_rlhf_objective(mb); obj.backward()
    nn.utils.clip_grad_norm_(self.model.parameters(), args.max_grad_norm)
    self.optimizer.step(); self.step += 1
self.scheduler.step()
```

Smoke test first: kl_coef=0.0, total_phases=30, warmup_steps=0 → expect "This is……"

APPENDIX

Lora · LoraHooks · TransformerWithValueHeadLora

```
# Lora.forward: force (x A) B, never A @ B
tmp = einops.einsum(x, self.A, "... inst d_in, inst d_in rank -> ... inst rank")
out = einops.einsum(tmp, self.B, "... inst rank, inst rank d_out -> ... inst d_out")
return out * self.lora_alpha / self.rank      # A: kaiming, B: zeros

# LoraHooks, per layer: cache inputs, then add the LoRA output
#   blocks.{i}.ln1.hook_normalized  -> cache input to q/k/v
#   blocks.{i}.attn.hook_q / _k / _v -> + lora_q / lora_k / lora_v
#   blocks.{i}.attn.hook_z          -> cache input to W_O
#   blocks.{i}.hook_attn_out        -> + lora_o, summed over heads

# TransformerWithValueHeadLora
fwd_hooks = self.lora_fwd_hooks + [self.value_head_hook]
self.ref_model = self.model      # hooks off = original GPT-2
```

n_kv_heads may be < n_qo_heads (grouped-query attention, e.g. Llama): use n_kv_heads for k and v.

APPENDIX

GrpoTrainer: what to delete

```
# rollout_phase: no compute_advantages
rewards_normed = normalize_reward(rewards)
advantages = einops.repeat(rewards_normed,
                           "b -> b g", g=gen_len)

# compute_rlhf_objective: no value loss
objective = clipped_surrogate_objective + entropy_bonus
return objective - kl_penalty

# model: TransformerWithLora (use_value_head=False)
# ref_model = model (LoRA hooks off)
```

Advantage

normalised reward, same on every token

Deleted

value head, value loss, compute_advantages

Group

all 128 samples share "This is"

Bonus ideas from the notebook

Adaptive KL
raise or lower kl_coef to hit a target KL

Frozen layers / differential LRs
parameter groups per layer; saves memory

Bigger models
gpt2-medium → xl: smaller minibatches

LoRA on the MLP
extra hooks on the MLP in and out

Mixed precision
bfloat16 adapters; LayerNorm stays float32

Learn a reward model
from human comparisons (Christiano 2017)

APPENDIX

Line up before slicing: the output at position i predicts token i+1

position	0	1	2	3	4	5	…	31	32
token	<BOS>	This	is	a	list	of	…	page	.
		prompt (3)				generated (30)			
output predicts	→ 1	→ 2	→ 3	→ 4	→ 5	→ 6	…	→ 32	(unused)

log-probs of the 30 generated tokens come from outputs at positions 2–31: `logits[:, prefix_len-1 : -1]`

values: V before each generated token = positions 2–31; V after = positions 3–32, with the **real reward** replacing the last.
advantage = after − before → shape (128, 30)

APPENDIX

Timing plan

part	full	skip amber	running (full)
1 · Revision	11.3	8.2	11
2 · Why RL	8.4	8.4	20
3 · Reward	14.2	11.7	34
4 · Agent	8.9	5.8	43
5 · Credit	12.8	10.3	56
6 · KL	24.6	17.3	80
7 · Train & LoRA	24.6	18.0	105
Opening + wrap-up	2.8	2.2	108
Total	**108 min**	**82 min**	

Estimates: ~0.55 min per slide + ~10 s per extra placard, 2.5 min per demo, 1.3 min per Slido question, 2 min per pause.